

REACT + DJANGO

Подключение React к Django-бэкенду

Полное пошаговое руководство
для проекта Amazone Clone

От пустой папки до работающего интернет-магазина:
Vite + TypeScript + Axios + Zustand + React Router

Версия 1.0 · Июнь 2026

898 тестов Django · 14 модулей · Все API готовы

Для Windows + Python 3.14 + Node.js 20+

Содержание

Часть 1. Архитектура: как React общается с Django

- 1.1 Что такое SPA и чем отличается от серверного рендеринга
 - 1.2 Архитектурная диаграмма запросов
 - 1.3 JWT-авторизация: как работает токенный доступ
 - 1.4 CORS: почему браузер блокирует запросы и как это исправить
 - 1.5 Vite proxy: зачем и как перенаправлять запросы
-

Часть 2. Полная карта API-эндпоинтов Django

- 2.1 Auth — регистрация, вход, обновление токена
 - 2.2 Users — профиль, адреса
 - 2.3 Catalog — товары, категории, бренды
 - 2.4 Cart — корзина
 - 2.5 Orders — заказы
 - 2.6 Payments — платежи
 - 2.7 Reviews — отзывы
 - 2.8 Discounts — купоны и скидки
 - 2.9 Shipping — доставка
 - 2.10 Wishlist — избранное
 - 2.11 Notifications — уведомления
 - 2.12 Analytics — аналитика
 - 2.13 Health — проверка бэкенда
-

Часть 3. Установка и настройка проекта

- 3.1 Установка Node.js и npm
 - 3.2 Создание проекта через Vite
 - 3.3 Установка зависимостей
 - 3.4 Настройка Vite proxy
 - 3.5 Структура папок проекта
-

Часть 4. API-клиент (axios + JWT)

- 4.1 Создание экземпляра axios
 - 4.2 Request interceptor — подстановка токена
 - 4.3 Response interceptor — авто-refresh при 401
 - 4.4 Файл endpoints.ts — все URL в одном месте
 - 4.5 Проверка: health-check из браузера
-

Часть 5. Глобальное состояние (Zustand)

- 5.1 Что такое Zustand и зачем он нужен
 - 5.2 authStore — пользователь, логин, регистрация
 - 5.3 cartStore — корзина, добавление, удаление
 - 5.4 wishlistStore — избранное
 - 5.5 notificationStore — уведомления
-

Часть 6. Роутинг (React Router)

- 6.1 Настройка маршрутов
 - 6.2 ProtectedRoute — защита страниц
 - 6.3 Layout — общий каркас страниц
-

Часть 7. Страницы авторизации

- 7.1 LoginPage — вход по email
 - 7.2 RegisterPage — регистрация
 - 7.3 Автоход при обновлении страницы
-

Часть 8. Каталог товаров

- 8.1 ProductListPage — список с фильтрами и поиском
 - 8.2 ProductDetailPage — карточка товара
-

Часть 9. Корзина

- 9.1 CartPage — полная страница корзины
 - 9.2 Боковая панель корзины (CartSidebar)
-

Часть 10. Оформление заказа

- 10.1 OrderCreatePage — выбор адреса, доставка, купон
 - 10.2 OrderDetailPage — детали заказа
 - 10.3 OrderListPage — список заказов
-

Часть 11. Личный кабинет

- 11.1 ProfilePage — редактирование профиля
 - 11.2 Управление адресами доставки
-

Часть 12. Избранное и уведомления

- 12.1 WishlistPage — список желаний
 - 12.2 NotificationBell — колокольчик уведомлений
-

Часть 13. Главная страница

- 13.1 HomePage — hero-баннер + популярные товары
-

Часть 14. TypeScript-типы для API

14.1 Полный файл types.ts

Часть 15. Тестирование и отладка

15.1 Проверка каждого эндпоинта вручную

15.2 Типичные ошибки и их решения

15.3 Django Debug Toolbar + React DevTools

Часть 16. Что делать дальше

16.1 Улучшения: Tailwind, MUI, react-query

16.2 Deploy: как выложить в интернет

16.3 Чек-лист готовности к production

Приложение А. Полная карта файлов проекта

Приложение В. Глоссарий (40 терминов)

Часть 1. Архитектура: как React общается с Django

1.1 Что такое SPA и чем отличается от серверного рендеринга

[DOC] [React: Quick Start](#) · [MDN: SPA](#)

SPA (Single Page Application) — это веб-приложение, которое загружает одну HTML-страницу, а потом динамически обновляет её содержимое без перезагрузки. React — это SPA-фреймворк.

Традиционный подход (Django templates):

Пользователь кликает «Каталог»

- Браузер отправляет GET /catalog/
- Django рендерит HTML на сервере
- Браузер ПОЛНОСТЬЮ перезагружает страницу
- Пользователь видит каталог

Время: 300–800мс (полная перезагрузка)
НЕТ Теряется состояние: скролл, открытые меню, введённый текст

SPA-подход (React):

Пользователь кликает «Каталог»

- React меняет URL на /products (БЕЗ перезагрузки!)
- React отправляет GET /api/v1/catalog/products/
- Django возвращает JSON (не HTML!)
- React рисует каталог из JSON

Время: 50–150мс (только данные, без HTML/CSS/JS)
ДА Состояние сохраняется: скролл, меню, текст

Аспект	Django Templates	React SPA
Кто рисует HTML?	Сервер (Python)	Браузер (JavaScript)
Что возвращает сервер?	Готовый HTML	Чистый JSON
Навигация	Полная перезагрузка	Без перезагрузки
Скорость	Медленнее	Быстрее
SEO	Отлично	Нужно SSR (Next.js)
Сложность	Проще	Сложнее (2 проекта)

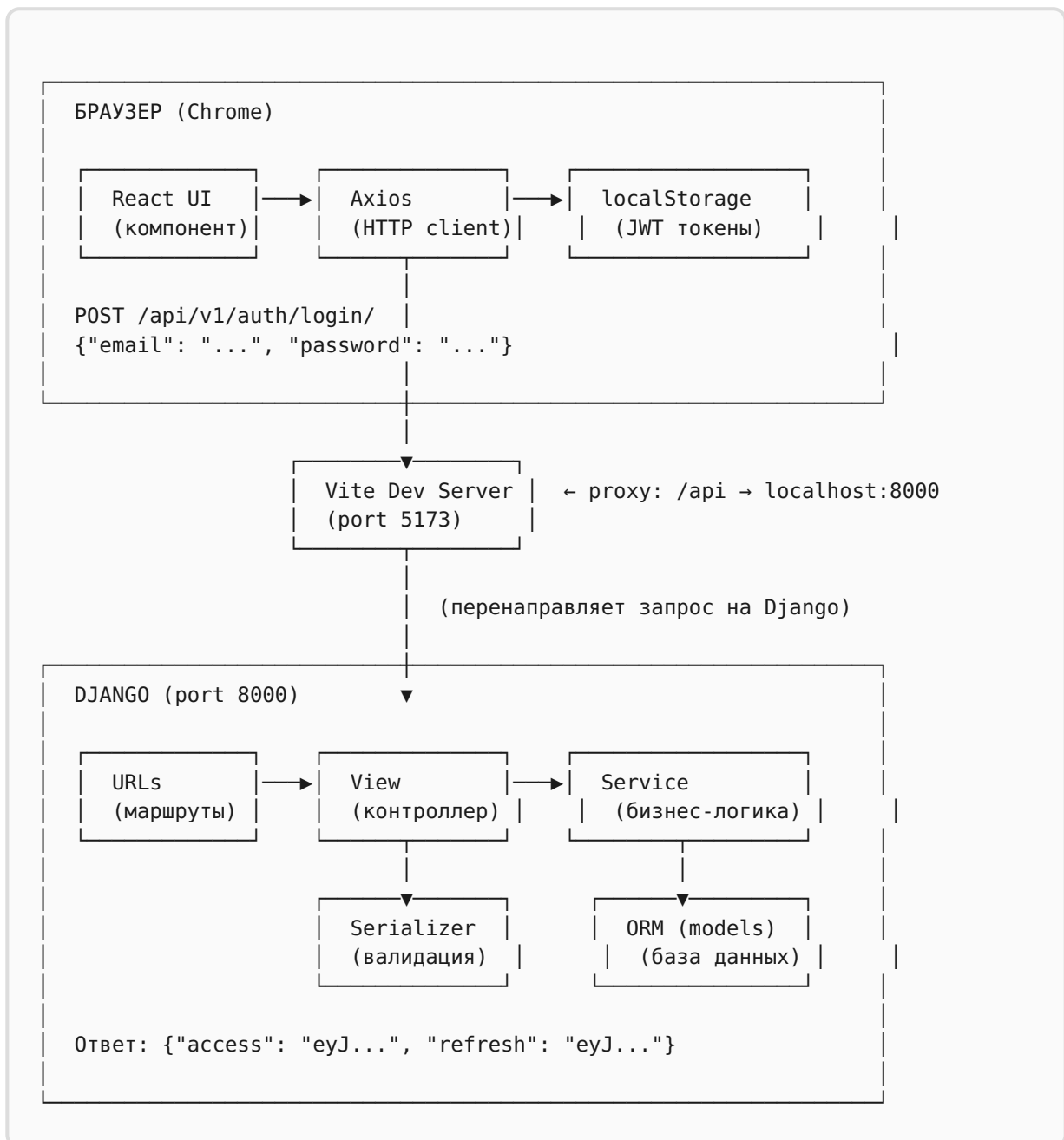
[!] Важно понимать

Django и React — это **два отдельных проекта**, которые общаются через HTTP-запросы (API). Django не знает о React, React не знает о Django. Единственная связь — это JSON, который летает между ними по сети.

1.2 Архитектурная диаграмма запросов

[DOC] [REST Architecture](#) · [DRF Quickstart](#)

Вот что происходит, когда пользователь нажимает «Войти» в React-приложении:

**Ключевые порты:**

Порт	Что работает	Для чего
localhost:5173	Vite + React	Фронтенд (браузер)
localhost:8000	Django + DRF	Бэкенд (API)

1.3 JWT-авторизация: как работает токенный доступ

[DOC] [SimpleJWT](#) · [jwt.io](#)

JWT (JSON Web Token) — это способ авторизации без сессий. Вместо хранения сессии на сервере, браузер хранит **токен** — зашифрованную строку, которая доказывает «я — этот пользователь».

Два токена

Токен	Время жизни	Где хранится	Зачем
access	15 минут	localStorage	Прикрепляется к каждому запросу
refresh	7 дней	localStorage	Получить новый access без повторного логина

Жизненный цикл JWT

1. ПОЛЬЗОВАТЕЛЬ ВВОДИТ EMAIL + ПАРОЛЬ
|
▼
2. REACT ОТПРАВЛЯЕТ POST /api/v1/auth/login/
{"email": "user@mail.com", "password": "secret123"}
|
▼
3. DJANGO ПРОВЕРЯЕТ ПАРОЛЬ → возвращает:
{"access": "eyJhbG...", "refresh": "eyJhbG..."}
|
▼
4. REACT СОХРАНЯЕТ ТОКЕНЫ В localStorage
localStorage.setItem('access_token', data.access)
localStorage.setItem('refresh_token', data.refresh)
|
▼
5. КАЖДЫЙ СЛЕДУЮЩИЙ ЗАПРОС:
GET /api/v1/users/me/
Header: Authorization: Bearer eyJhbG...
|
▼
6. ЕСЛИ ACCESS УСТАРЕЛ (15 мин) → сервер вернёт 401
|
▼
7. REACT АВТОМАТИЧЕСКИ ОТПРАВЛЯЕТ:
POST /api/v1/auth/refresh/
{"refresh": "eyJhbG..."}
|
▼
8. DJANGO ВЫДАЁТ НОВЫЙ ACCESS:
{"access": "eyJNEW..."}
|
▼
9. REACT ПОВТОРЯЕТ ОРИГИНАЛЬНЫЙ ЗАПРОС С НОВЫМ ТОКЕНОМ
(пользователь ничего не замечает!)

⚠ Почему два токена?

Если хакер украдёт `access` — он сможет пользоваться им только 15 минут. `refresh` используется редко (раз в 15 мин), его легче защитить. Это как с банковской картой: основная карта (`access`) — для ежедневных покупок, а запасная (`refresh`) — хранится дома и используется только для перевыпуска основной.

Что хранится внутри JWT-токена?

```
// Access token (расшифрованный):
{
  "token_type": "access",
  "exp": 1718300000,      // когда истекает (Unix timestamp)
  "iat": 1718299100,      // когда создан
  "jti": "abc123...",     // уникальный ID токена
  "user_id": 42           // ID пользователя в Django
}

// Django проверяет: токен не истёк? подпись верна? → user_id = 42
```

1.4 CORS: почему браузер блокирует запросы и как это исправить

[\[DOC\] django-cors-headers](#) · [MDN: CORS](#)

CORS (Cross-Origin Resource Sharing) — это механизм безопасности браузера. По умолчанию браузер **запрещает** JavaScript-коду с одного домена делать запросы к другому домену.

БЕЗ CORS:

React (localhost:5173) —XHR→ Django (localhost:8000)
 |
 НЕТ Браузер блокирует! Разные порты = разные origins!

С CORS:

React (localhost:5173) —XHR→ Django (localhost:8000)
 |
 Django отвечает с заголовком:
 Access-Control-Allow-Origin: http://localhost:5173
 |
 ДА Браузер видит заголовок → разрешает!

Как CORS настроен в нашем Django

```
# config/settings.py

# 1. Приложение corsheaders добавлено в INSTALLED_APPS
INSTALLED_APPS = [
    ...
    "corsheaders", # ← Это!
]

# 2. Middleware ДО CommonMiddleware
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware", # ← ДО!
    "django.middleware.common.CommonMiddleware",
    ...
]

# 3. Настройки CORS
CORS_ALLOW_ALL_ORIGINS = True # В DEBUG = True (разрешить всем)
CORS_ALLOWED_ORIGINS = [
    "http://localhost:3000", # CRA
    "http://localhost:5173", # Vite ← наш фронтенд
]
CORS_ALLOW_CREDENTIALS = True # Разрешить куки + JWT-заголовки
CORS_ALLOW_HEADERS = [
    "authorization", # ← JWT-токен!
    "content-type",
    ...
]
```

[!!!] Частая ошибка

Если `CorsMiddleware` стоит ПОСЛЕ `CommonMiddleware`, CORS не работает!
Порядок `middleware` в Django **имеет значение**. Наш проект уже настроен правильно — не меняй порядок.

1.5 Vite проху: зачем и как перенаправлять запросы

[\[DOC\] Vite: Server Options](#)

Без проху тебе пришлось бы писать полный URL в каждом запросе:

```
// НЕТ Без проху – приходится указывать полный URL
axios.get('http://localhost:8000/api/v1/catalog/products/')
```

С proxy Vite автоматически перенаправляет запросы:

```
// ДА С proxy — короткий и красивый URL
axios.get('/api/v1/catalog/products/')
// Vite сам подставит http://localhost:8000
```

Как это работает



ДА Proxy работает ТОЛЬКО в dev-режиме

Когда ты запускаешь `npm run dev`, Vite проксирует запросы. В production (после `npm run build`) proxy не работает — nginx должен сам перенаправлять `/api/*` на Django.

Часть 2. Полная карта API-эндпоинтов Django

Прежде чем писать React, нужно точно знать, какие эндпоинты доступны на бэкенде. Ниже — полный список всех URL, методов, запросов и ответов.

[DOC] Документация в браузере

Запусти `python manage.py runserver` и открой **`http://localhost:8000/api/v1/docs/`** — там интерактивная Swagger-документация, где можно тестировать каждый эндпоинт прямо в браузере.

2.1 Auth — регистрация, вход, обновление токена

Эти эндпоинты **публичные** (не требуют JWT).

Метод	URL	Тело запроса	Ответ
POST	<code>/api/v1/auth/register/</code>	<pre>{ "email": "user@mail.com", "username": "cooluser", "password": "Secret123!", "password_confirm": "Secret123!", "first_name": "Иван", "last_name": "Петров" }</pre>	<pre>{ "id": 1, "email": "user@mail.com", "username": "cooluser", "first_name": "Иван", "last_name": "Петров" }</pre>
POST	<code>/api/v1/auth/login/</code>	<pre>{ "email": "user@mail.com", "password": "Secret123!" }</pre>	<pre>{ "access": "eyJhbG...", "refresh": "eyJhbG..." }</pre>
POST	<code>/api/v1/auth/refresh/</code>	<pre>{ "refresh": "eyJhbG..." }</pre>	<pre>{ "access": "eyJNEW...", "refresh": "eyJNEW2..." // при ROTATE= }</pre>
POST	<code>/api/v1/auth/change-password/</code>	<pre>{ "old_password": "Old123!", "new_password": "New456!", "new_password_confirm": "New456!" }</pre>	<pre>{ "detail": "Password changed" }</pre>

2.2 Users — профиль, адреса

Требуют JWT в заголовке `Authorization: Bearer <access>`

Метод	URL	Описание
GET	<code>/api/v1/users/me/</code>	Получить профиль
PATCH	<code>/api/v1/users/me/</code>	Обновить профиль (<code>first_name</code> , <code>last_name</code> , <code>phone</code>)
DELETE	<code>/api/v1/users/me/</code>	Удалить аккаунт
GET	<code>/api/v1/users/addresses/</code>	Список адресов
POST	<code>/api/v1/users/addresses/</code>	Создать адрес
GET	<code>/api/v1/users/addresses/{id}/</code>	Детали адреса
PATCH	<code>/api/v1/users/addresses/{id}/</code>	Обновить адрес
DELETE	<code>/api/v1/users/addresses/{id}/</code>	Удалить адрес
POST	<code>/api/v1/users/addresses/{id}/default/</code>	Сделать основным

Пример: GET `/api/v1/users/me/`

```
// Request:
GET /api/v1/users/me/
Authorization: Bearer eyJhbG...

// Response (200):
{
  "id": 1,
  "email": "user@mail.com",
  "username": "cooluser",
  "first_name": "Иван",
  "last_name": "Петров",
  "phone": "+79991234567",
  "profile": {
    "timezone": "Europe/Moscow",
    "language": "ru",
    "avatar": "/media/avatars/photo.jpg"
  }
}
```

2.3 Catalog — товары, категории, бренды

Публичные (AllowAny).

Метод	URL	Описание
GET	/api/v1/catalog/products/	Список товаров (фильтры, поиск, пагинация)
POST	/api/v1/catalog/products/create/	Создать товар (staff)
GET	/api/v1/catalog/products/{slug_or_uuid}/	Карточка товара
PATCH	/api/v1/catalog/products/{uuid}/update/	Обновить товар (staff)
GET	/api/v1/catalog/categories/	Дерево категорий
GET	/api/v1/catalog/categories/{slug}/	Детали категории
GET	/api/v1/catalog/brands/	Список брендов
GET	/api/v1/catalog/brands/{slug}/	Детали бренда

Фильтры для GET /api/v1/catalog/products/

GET /api/v1/catalog/products/?search=iphone	# Поиск по названию/описанию
GET /api/v1/catalog/products/?category=smartfony	# Фильтр по категории (slug)
GET /api/v1/catalog/products/?brand=apple	# Фильтр по бренду (slug)
GET /api/v1/catalog/products/?min_price=10000	# Минимальная цена
GET /api/v1/catalog/products/?max_price=50000	# Максимальная цена
GET /api/v1/catalog/products/?ordering=-rating	# Сортировка: -rating, price, -price, name
GET /api/v1/catalog/products/?page=2	# Пагинация (20 на страницу)
GET /api/v1/catalog/products/?is_in_stock=true	# Только в наличии

Пример: ответ списка товаров

```
{
  "count": 156,
  "next": "http://localhost:8000/api/v1/catalog/products/?page=2",
  "previous": null,
  "results": [
    {
      "id": "uuid-string",
      "name": "iPhone 15 Pro",
      "slug": "iphone-15-pro",
      "description": "Флагманский смартфон Apple...",
      "min_price": "89990.00",
      "max_price": "149990.00",
      "rating": "4.70",
      "reviews_count": 142,
      "views_count": 5832,
      "main_image": "/media/products/iphone15.jpg",
      "brand": {
        "name": "Apple",
        "slug": "apple"
      },
      "category": {
        "name": "Смартфоны",
        "slug": "smartfony"
      }
    }
  ]
}
```

2.4 Cart — корзина

AllowAny (гости по session_key, авторизованные по JWT).

Метод	URL	Описание
GET	/api/v1/cart/	Получить корзину
DELETE	/api/v1/cart/	Очистить корзину
POST	/api/v1/cart/items/	Добавить товар {variant_id, quantity}
PATCH	/api/v1/cart/items/{id}/	Изменить количество {quantity}
DELETE	/api/v1/cart/items/{id}/	Удалить позицию
POST	/api/v1/cart/merge/	Слить гостевую корзину {session_key}

Пример: POST /api/v1/cart/items/

```
// Request:
{
  "variant_id": 5,
  "quantity": 2
}

// Response (200):
{
  "id": 1,
  "items": [
    {
      "id": 10,
      "variant_id": 5,
      "product_name": "iPhone 15 Pro 256GB",
      "sku": "IP15P-256-BLK",
      "unit_price": "89990.00",
      "quantity": 2,
      "total_price": "179980.00",
      "image": "/media/products/iphone15.jpg"
    }
  ],
  "total_quantity": 2,
  "total_price": "179980.00"
}
```

2.5 Orders — заказы

Требуют JWT. `order_number` — публичный ID (формат: `ORD-000001`).

Метод	URL	Описание
GET	/api/v1/orders/	Список заказов
POST	/api/v1/orders/	Оформить заказ из корзины
GET	/api/v1/orders/{order_number}/	Детали заказа
PATCH	/api/v1/orders/{order_number}/status/	Изменить статус (staff)
POST	/api/v1/orders/{order_number}/cancel/	Отменить заказ

Статус-машина заказа (FSM)

```
PENDING → CONFIRMED → PROCESSING → SHIPPED → DELIVERED
  |           |
  ▼           ▼
CANCELLED   CANCELLED
```

Пример: POST /api/v1/orders/

```
// Request:
{
  "address_id": 1,
  "shipping_method_id": 2,
  "comment": "Позвонить перед доставкой"
}

// Response (201):
{
  "order_number": "ORD-000042",
  "status": "PENDING",
  "items": [
    {
      "variant_id": 5,
      "product_name": "iPhone 15 Pro 256GB",
      "unit_price": "89990.00",
      "quantity": 2,
      "total_price": "179980.00"
    }
  ],
  "subtotal": "179980.00",
  "delivery_cost": "350.00",
  "discount": "0.00",
  "total": "180330.00",
  "created_at": "2026-06-14T12:00:00Z"
}
```

2.6 Payments — платежи

Метод	URL	Описание
GET	/api/v1/payments/	Список платежей
POST	/api/v1/payments/	Создать платёж {order, payment_method}
GET	/api/v1/payments/{payment_number}/	Детали платежа
POST	/api/v1/payments/webhook/	Вебхук от платёжной системы
POST	/api/v1/payments/{payment_number}/refund/	Возврат (staff)
POST	/api/v1/payments/{payment_number}/cancel/	Отмена платежа

2.7 Reviews — отзывы

Метод	URL	Описание
GET	/api/v1/reviews/?product={uuid}	Список отзывов товара
POST	/api/v1/reviews/	Создать отзыв {product, rating, text}
GET	/api/v1/reviews/{id}/	Детали отзыва
PATCH	/api/v1/reviews/{id}/	Обновить отзыв
DELETE	/api/v1/reviews/{id}/	Удалить отзыв

2.8 Discounts — купоны и скидки

Метод	URL	Описание
GET	/api/v1/discounts/coupons/	Список купонов (staff)
POST	/api/v1/discounts/apply/	Применить купон {order_id, code}
POST	/api/v1/discounts/remove/	Убрать купон {order_id}
POST	/api/v1/discounts/preview/	Предпросмотр скидки {order_id, code}

2.9 Shipping — доставка

Метод	URL	Описание
GET	/api/v1/shipping/methods/	Способы доставки
POST	/api/v1/shipping/calculate/	Расчёт стоимости {address_id, items}
GET	/api/v1/shipping/shipments/	Список отправлений
POST	/api/v1/shipping/shipments/create/	Создать отправление (staff)
GET	/api/v1/shipping/shipments/{id}/	Детали отправления
PATCH	/api/v1/shipping/shipments/{id}/status/	Изменить статус (staff)
POST	/api/v1/shipping/shipments/{id}/tracking/	Добавить трек-номер (staff)
GET	/api/v1/shipping/track/{tracking}/	Отслеживание (public)

2.10 Wishlist — избранное

Требуют JWT.

Метод	URL	Описание
GET	/api/v1/wishlist/	Мой список желаний
POST	/api/v1/wishlist/add/	Добавить товар {variant_id}
DELETE	/api/v1/wishlist/remove/{item_id}/	Удалить из избранного
POST	/api/v1/wishlist/move-to-cart/	Переместить в корзину {item_id}
DELETE	/api/v1/wishlist/clear/	Очистить весь список

2.11 Notifications — уведомления

Требуют JWT.

Метод	URL	Описание
GET	/api/v1/notifications/	Список уведомлений
GET	/api/v1/notifications/unread/	Непрочитанные
GET	/api/v1/notifications/unread-count/	Количество непрочитанных
POST	/api/v1/notifications/read-all/	Прочитать все
POST	/api/v1/notifications/{id}/read/	Прочитать одно

2.12 Analytics — аналитика

Только для администраторов (IsAdminUser).

Метод	URL	Описание
GET	/api/v1/analytics/dashboard/	Комплексный дашборд
GET	/api/v1/analytics/sales/	Сводка продаж
GET	/api/v1/analytics/sales/timeline/	График продаж
GET	/api/v1/analytics/top-products/	Топ товаров
GET	/api/v1/analytics/top-categories/	Топ категорий
GET	/api/v1/analytics/top-customers/	Топ покупателей
GET	/api/v1/analytics/conversion/	Конверсия
GET	/api/v1/analytics/most-viewed/	Самые просматриваемые

2.13 Health — проверка бэкенда

Публичный, без авторизации.

```
GET /api/v1/health/

// Response (200):
{
  "status": "ok",
  "version": "1.0.0",
  "database": "ok"
}
```

[!] Совет

Используй `GET /api/v1/health/` в React для проверки, что бэкенд запущен. Если ответ `{"status": "ok"}` — всё работает. Если ошибка — покажи пользователю «Сервер недоступен».

Часть 3. Установка и настройка проекта

3.1 Установка Node.js и npm

[DOC] [Node.js](#) · [npm docs](#)

React — это JavaScript-библиотека. Чтобы писать на JS, нужен **Node.js** (среда выполнения) и **npm** (менеджер пакетов — как `pip` для Python).

Как проверить, установлен ли Node.js

```
# Открой терминал (cmd / PowerShell) и введи:
node --version
# Должно быть: v20.x.x или выше

npm --version
# Должно быть: 10.x.x или выше
```

Если не установлен

```
# Скачай с официального сайта:
# https://nodejs.org/
# Выбери LTS-версию (Long Term Support) — стабильная

# После установки перезапусти терминал и проверь:
node --version    # → v20.15.0
npm --version     # → 10.7.0
```

⚠ Версия Node.js

Нужна версия **18+ или 20+**. Если у тебя более старая — обнови. Vite и React 18+ не работают на Node 16.

3.2 Создание проекта через Vite

[DOC] [Vite: Getting Started](#)

Vite (читается «вит», фр. «быстрый») — это инструмент сборки, который заменяет Create React App (CRA). Он в 10–100 раз быстрее.

Почему Vite, а не CRA?

Аспект	Create React App	Vite
Запуск dev-сервера	10–30 секунд	0.3–1 секунда
Hot Module Replacement	1–3 секунды	мгновенно
Поддержка	Прекращена (2023)	Активная
Конфигурация	Скрыта (webpack)	Простая (vite.config.ts)

1 Создание проекта

```
# Открой терминал и перейди на уровень ВЫШЕ папки проекта:
cd I:\NewPythonProjects

# Создай React-проект с TypeScript:
npm create vite@latest frontend -- --template react-ts

# Перейди в папку:
cd frontend

# Установи базовые зависимости:
npm install
```

Что создалось:

```

I:\NewPythonProjects\
├── Amazon_Clone/      ← Django-бэкенд (уже есть)
│   ├── config/
│   ├── apps/
│   └── manage.py
└── frontend/         ← React-фронтенд (новый!)
    ├── src/
    │   ├── App.tsx    ← Главный компонент
    │   ├── App.css
    │   ├── main.tsx   ← Точка входа
    │   ├── index.css
    │   └── vite-env.d.ts
    ├── index.html
    ├── package.json
    ├── tsconfig.json
    └── vite.config.ts

```

2 Проверка: запускаем пустой проект

```

# В папке frontend:
npm run dev

# Вывод:
# VITE v5.x.x ready in 300ms
# → Local: http://localhost:5173/
# → Network: http://192.168.x.x:5173/

```

Открой **http://localhost:5173** — увидишь страницу с логотипом Vite + React.

Останови сервер: `Ctrl + C`

3.3 Установка зависимостей

[DOC] [Axios docs](#) · [React Router](#) · [Zustand](#)

3 Устанавливаем все нужные пакеты

```

# В папке frontend:
npm install axios react-router-dom zustand

```

Что делает каждый пакет

Пакет	Версия	Зачем	Аналог в Python
axios	^1.7	HTTP-запросы к Django API. Умеет интерцепторы (автоподстановка JWT).	requests
react-router-dom	^6.x	Навигация между страницами без перезагрузки (SPA-роутинг).	Django <code>urls.py</code>
zustand	^4.x	Глобальное состояние (пользователь, корзина). Проще Redux.	Django <code>request.user</code>

Аналогия с Python/Django

React	Django	Что делает
axios	requests / httpx	HTTP-запросы
react-router-dom	urls.py	Маршрутизация
zustand	request.user / сессия	Состояние пользователя
localStorage	django.contrib.sessions	Хранение данных в браузере
react	Django templates	Рендеринг UI
TypeScript	мypy + type hints	Проверка типов
npm	pip	Менеджер пакетов
package.json	requirements.txt	Список зависимостей
vite.config.ts	settings.py	Конфигурация проекта

3.4 Настройка Vite proxy

[\[DOC\] Vite config](#)

Proxy — это «пересылка» запросов. Когда React делает запрос на `/api/v1/...`, Vite автоматически пересылает его на Django (`localhost:8000`).

4 Замени содержимое `vite.config.ts`

```
// vite.config.ts
// Конфигурация сборщика Vite.
// проху – перенаправляет API-запросы на Django.

import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  // Плагин для React (JSX-трансформация)
  plugins: [react()],

  server: {
    // Порт фронтенд-сервера
    port: 5173,

    // Проху – автоматическое перенаправление запросов
    proxy: {
      // Все запросы, начинающиеся с /api → пересылаются на Django
      '/api': {
        target: 'http://localhost:8000', // Адрес Django
        changeOrigin: true, // Подменять Host-заголовок
        // НЕ пишем rewrite – пути совпадают!
      },

      // Media-файлы (картинки товаров, аватары)
      '/media': {
        target: 'http://localhost:8000',
        changeOrigin: true,
      },
    },
  },
})
```

Почему это работает

```

React: axios.get('/api/v1/catalog/products/')
      |
      ▼
Vite (port 5173):  видит /api → проху
      |
      ▼
Django (port 8000): GET /api/v1/catalog/products/
      |
      ▼
Django возвращает JSON
      |
      ▼
Vite пересылает JSON в React ДА

```

⚠ **Django должен быть запущен!**

Proxy только перенаправляет запросы. Если Django не запущен (`python manage.py runserver`), Vite вернёт ошибку `ECONNREFUSED` .

3.5 Структура папок проекта

Мы будем создавать файлы в определённой структуре. Вот итоговая карта:

```

frontend/
├── public/                                ← Статичные файлы (favicon, иконки)
│   └── vite.svg
├── src/
│   ├── api/                              ← Слой общения с бэкендом
│   │   ├── client.ts                    ← axios-экземпляр + JWT-интерцепторы
│   │   └── endpoints.ts                 ← Все URL в одном месте
│   ├── stores/                          ← Глобальное состояние (Zustand)
│   │   ├── authStore.ts                 ← Пользователь, логин, регистрация
│   │   ├── cartStore.ts                 ← Корзина
│   │   ├── wishlistStore.ts             ← Избранное
│   │   └── notificationStore.ts         ← Уведомления
│   ├── types/                           ← TypeScript-типы
│   │   └── index.ts                     ← Все интерфейсы для API-ответов
│   ├── components/                      ← Переиспользуемые компоненты
│   │   ├── Layout.tsx                  ← Общий каркас (хедер + футер)
│   │   ├── ProtectedRoute.tsx          ← Защита страниц от неавторизованных
│   │   ├── CartSidebar.tsx             ← Боковая панель корзины
│   │   └── NotificationBell.tsx        ← Колокольчик уведомлений
│   ├── pages/                           ← Страницы (каждая = один экран)
│   │   ├── HomePage.tsx                ← /
│   │   ├── LoginPage.tsx               ← /login
│   │   ├── RegisterPage.tsx            ← /register
│   │   ├── ProductListPage.tsx         ← /products
│   │   ├── ProductDetailPage.tsx       ← /products/:slug
│   │   ├── CartPage.tsx                 ← /cart
│   │   ├── OrderCreatePage.tsx         ← /checkout
│   │   ├── OrderListPage.tsx           ← /orders
│   │   ├── OrderDetailPage.tsx         ← /orders/:orderNumber
│   │   ├── ProfilePage.tsx             ← /profile
│   │   ├── WishListPage.tsx            ← /wishlist
│   │   └── NotFoundPage.tsx            ← 404
│   ├── router.tsx                       ← Все маршруты
│   ├── main.tsx                         ← Точка входа (подключает RouterProvider)
│   ├── App.tsx                          ← (можно удалить – заменён на router)
│   └── index.css                         ← Глобальные стили
├── index.html                           ← HTML-шаблон (Vite подключает JS сюда)
├── package.json                         ← Зависимости и скрипты
├── tsconfig.json                        ← Настройки TypeScript
├── tsconfig.app.json
├── tsconfig.node.json
└── vite.config.ts                       ← Настройки Vite + proxy

```

[!] Почему так, а не иначе?

api/ — весь код, общающийся с бэкендом, в одном месте. Если API изменится — меняешь только тут.

stores/ — глобальное состояние. Если нужно «поделиться» данными между страницами (корзина, пользователь) — используем store.

pages/ — каждая страница = отдельный файл. Легко найти нужный экран.

components/ — маленькие переиспользуемые кусочки UI (кнопки, формы, модалки).

types/ — TypeScript-типы. Если бэкенд возвращает `{"id": 1, "name": "..."}` , описываем это как `interface Product { id: number; name: string; }` .

[!!] Не пиши весь код в App.tsx!

Самая частая ошибка новичков — написать всё в одном файле. Через 2 недели ты не сможешь разобраться в 2000 строках кода. Разделяй на файлы и папки с первого дня.

Часть 4. API-клиент (axios + JWT)

Это самый важный файл всего фронтенда. Он отвечает за ВСЕ запросы к Django. Если он правильно настроен, остальные файлы просто вызывают `api.get(...)` и `api.post(...)` — без мыслей о токенах.

4.1 Создание экземпляра axios

[DOC] [Axios Instance](#) · [Axios Interceptors](#)

5 Создай файл `src/api/client.ts`


```

// src/api/client.ts
// Единый HTTP-клиент для всех запросов к Django-бэкенду.
//
// ЧТО ДЕЛАЕТ ЭТОТ ФАЙЛ:
// 1. Создаёт экземпляр axios с базовым URL /api/v1
// 2. Автоматически подставляет JWT-токен в каждый запрос (request interceptor)
// 3. При получении 401 (токен истёк) – автоматически обновляет токен
//    и повторяет запрос (response interceptor)
//
// КАК ИСПОЛЬЗОВАТЬ:
// import api from '../api/client';
// const response = await api.get('/catalog/products/');
// const response = await api.post('/auth/login/', { email, password });
//
// ПОЧЕМУ НЕ ПРОСТО axios.get():
// - Нужно вручную добавлять Authorization-заголовок в КАЖДЫЙ запрос
// - Нет авто-refresh при истёкшем токене
// - Нет единого базового URL

import axios from 'axios';

// Базовый URL – все запросы будут начинаться с /api/v1/
// Vite проху перенаправит /api/v1/* → http://localhost:8000/api/v1/*
const API_BASE = '/api/v1';

// — СОЗДАЁМ ЭКЗЕМПЛЯР AXIOS —
// Зачем create(), а не просто axios?
// Чтобы настройки (baseUrl, заголовки) применялись ко ВСЕМ запросам.
const api = axios.create({
  baseUrl: API_BASE,
  headers: {
    'Content-Type': 'application/json',
  },
});

// — REQUEST INTERCEPTOR: подставляем JWT —
// Интерцептор = функция, которая ВСТРАИВАЕТСЯ между вызовом api.get()
// и фактической отправкой запроса.
//
// Схема:
// api.get('/users/me/')
//   → interceptor добавляет Authorization-заголовок
//   → запрос отправляется на сервер
//
// Без интерцептора пришлось бы писать:
// api.get('/users/me/', {
//   headers: { Authorization: `Bearer ${token}` } // КАЖДЫЙ РАЗ!
// })

api.interceptors.request.use(
  (config) => {
    // Читаем access-токен из localStorage
    const token = localStorage.getItem('access_token');
  }
);

```

```

    if (token) {
      // Добавляем заголовок: Authorization: Bearer eyJhbG...
      config.headers.Authorization = `Bearer ${token}`;
    }

    // Обязательно возвращаем config – иначе запрос не отправится!
    return config;
  },
  (error) => {
    // Если ошибка Д0 отправки запроса (маловероятно, но бывает)
    return Promise.reject(error);
  }
);

// — RESPONSE INTERCEPTOR: авто-refresh при 401 —
// Если сервер вернул 401 Unauthorized – значит access-токен истёк (15 мин).
// Мы автоматически:
//   1. Отправляем refresh-токен на /auth/refresh/
//   2. Получаем новый access-токен
//   3. Повторяем оригинальный запрос с новым токеном
// Пользователь ничего не замечает!

api.interceptors.response.use(
  // Если ответ успешный (2xx) – просто пропускаем
  (response) => response,

  // Если ошибка (4xx, 5xx) – проверяем, не истёк ли токен
  async (error) => {
    const originalRequest = error.config;

    // Проверяем: это 401 И мы ещё НЕ пробовали refresh
    // _retry – наш флаг, чтобы не заикнуться
    if (
      error.response?.status === 401 &&
      !originalRequest._retry &&
      !originalRequest.url?.includes('/auth/login/') // Не refresh'им при логине!
    ) {
      originalRequest._retry = true; // Помечаем: уже пробуем refresh

      const refreshToken = localStorage.getItem('refresh_token');

      if (refreshToken) {
        try {
          // Запрашиваем новый access-токен
          // Используем ПРЯМОЙ axios (не api!), чтобы не заикнуть интерцептор
          const { data } = await axios.post(`${API_BASE}/auth/refresh/`, {
            refresh: refreshToken,
          });

          // Сохраняем новый access-токен
          localStorage.setItem('access_token', data.access);

```

```

    // Если бэкэнд вернул новый refresh (ROTATE_REFRESH_TOKENS=True)
    if (data.refresh) {
      localStorage.setItem('refresh_token', data.refresh);
    }

    // Обновляем заголовок в оригинальном запросе
    originalRequest.headers.Authorization = `Bearer ${data.access}`;

    // Повторяем оригинальный запрос с новым токеном
    return api(originalRequest);

  } catch (refreshError) {
    // Refresh-токен тоже истёк (7 дней) – полный логアウト
    localStorage.removeItem('access_token');
    localStorage.removeItem('refresh_token');

    // Перенаправляем на страницу входа
    window.location.href = '/login';
    return Promise.reject(refreshError);
  }
} else {
  // Нет refresh-токена – пользователь не авторизован
  window.location.href = '/login';
}
}

// Для всех остальных ошибок – просто передаём дальше
return Promise.reject(error);
}
);

// Экспортируем – все остальные файлы будут использовать api.get(), api.post()
export default api;

```

[!!!] Почему `axios.post()`, а не `api.post()` при refresh?

В строке `const { data } = await axios.post(`${API_BASE}/auth/refresh/`, ...)` мы используем **прямой** `axios`, а не наш `api`. Причина: если использовать `api`, сработает request-интерцептор, который подставит старый (истёкший) access-токен. А нам нужно отправить ТОЛЬКО refresh-токен в теле, без access-заголовка.

4.2 Request interceptor — подстановка токена (подробно)

[\[DOC\] Axios Interceptors](#) · [localStorage API](#)

Разберём по шагам, что происходит при каждом запросе:

```
Вызов: api.get('/users/me/')
|
▼
① axios.create подставляет baseUrl:
GET /api/v1/users/me/
|
▼
② Request interceptor срабатывает:
Читает localStorage.getItem('access_token')
→ 'eyJhbGciOiJIUzI1NiIs...'
|
▼
③ Добавляет заголовок:
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
|
▼
④ Запрос отправляется на сервер:
GET /api/v1/users/me/
Host: localhost:8000 (через Vite proxy)
Authorization: Bearer eyJhbG...
Content-Type: application/json
```

4.3 Response interceptor — авто-refresh (подробно)

```

Сервер вернул 401 (access-токен истёк через 15 мин)
|
▼
① Проверяем: status === 401? → ДА
② Проверяем: _retry === false? → ДА (ещё не пробовали)
③ Проверяем: это НЕ /auth/login/? → ДА
|
▼
④ Читаем refresh_token из localStorage
|
▼
⑤ POST /api/v1/auth/refresh/
  {"refresh": "eyJhbG...старый_refresh"}
|
▼
⑥ Django проверяет refresh-токен → возвращает:
  {"access": "eyJhbG...НОВЫЙ_access",
   "refresh": "eyJhbG...НОВЫЙ_refresh"}
|
▼
⑦ Сохраняем новые токены в localStorage
|
▼
⑧ Обновляем заголовок в оригинальном запросе:
  Authorization: Bearer eyJhbG...НОВЫЙ_access
|
▼
⑨ Повторяем оригинальный запрос:
  api(originalRequest) → GET /api/v1/users/me/ с новым токеном
|
▼
⑩ Сервер возвращает 200 ДА
  Пользователь НЕ ЗАМЕТИЛ обновления токена!

```

4.4 Файл endpoints.ts — все URL в одном месте

6 Создай файл src/api/endpoints.ts

```

// src/api/endpoints.ts
// Все пути к API-эндпоинтам в одном файле.
//
// ПОЧЕМУ НЕ ПИСАТЬ URL ПРЯМО В КОДЕ:
//   Если на бэкенде изменится URL (например, /cart/ → /shopping-cart/),
//   придётся искать и менять его в 20 местах.
//   С endpoints.ts — меняем только здесь, в одном месте.
//
// КАК ИСПОЛЬЗОВАТЬ:
//   import { AUTH, CATALOG } from '../api/endpoints';
//   api.post(AUTH.LOGIN, { email, password });
//   api.get(CATALOG.PRODUCTS, { params: { search: 'iphone' } });
//   api.get(CATALOG.PRODUCT('iphone-15-pro'));
//
// as const — TypeScript-трюк: делает строки литеральными типами,
// чтобы автодополнение работало идеально.

// =====
// Auth — публичные эндпоинты
// =====
export const AUTH = {
  LOGIN: '/auth/login/', // POST {email, password} → {access, refresh}
  REGISTER: '/auth/register/', // POST {email, username, password, password_confirm}
  REFRESH: '/auth/refresh/', // POST {refresh} → {access}
  CHANGE_PASSWORD: '/auth/change-password/', // POST {old_password, new_password, new_password_confirm}
} as const;

// =====
// Users — профиль и адреса (требуют JWT)
// =====
export const USER = {
  ME: '/users/me/', // GET/PATCH/DELETE
  ADDRESSES: '/users/addresses/', // GET/POST
  ADDRESS: (id: number) => `/users/addresses/${id}/`, // GET/PATCH/DELETE
  ADDRESS_DEFAULT: (id: number) => `/users/addresses/${id}/default/`, // POST
} as const;

// =====
// Catalog — товары, категории, бренды (публичные)
// =====
export const CATALOG = {
  PRODUCTS: '/catalog/products/', // GET (list)
  PRODUCT_CREATE: '/catalog/products/create/', // POST (staff)
  PRODUCT: (slugOrUuid: string) => `/catalog/products/${slugOrUuid}/`, // GET
  PRODUCT_UPDATE: (uuid: string) => `/catalog/products/${uuid}/update/`, // PATCH (staff)
  CATEGORIES: '/catalog/categories/', // GET (tree)
  CATEGORY: (slug: string) => `/catalog/categories/${slug}/`, // GET
  BRANDS: '/catalog/brands/', // GET (list)
  BRAND: (slug: string) => `/catalog/brands/${slug}/`, // GET
} as const;

// =====
// Cart — корзина (AllowAnonymous)

```

```

// =====
export const CART = {
  CART: '/cart/', // GET/DELETE
  ITEMS: '/cart/items/', // POST (добавить)
  ITEM: (id: number) => `/cart/items/${id}/`, // PATCH/DELETE
  MERGE: '/cart/merge/', // POST (слить гостевую)
} as const;

// =====
// Orders – заказы (требуют JWT)
// =====
export const ORDER = {
  LIST: '/orders/', // GET/POST
  DETAIL: (orderNumber: string) => `/orders/${orderNumber}/`, // GET
  STATUS: (orderNumber: string) => `/orders/${orderNumber}/status/`, // PATCH (staff)
  CANCEL: (orderNumber: string) => `/orders/${orderNumber}/cancel/`, // POST
} as const;

// =====
// Payments – платежи
// =====
export const PAYMENT = {
  LIST: '/payments/', // GET/POST
  WEBHOOK: '/payments/webhook/', // POST
  DETAIL: (paymentNumber: string) => `/payments/${paymentNumber}/`, // GET
  REFUND: (paymentNumber: string) => `/payments/${paymentNumber}/refund/`, // POST (staff)
  CANCEL: (paymentNumber: string) => `/payments/${paymentNumber}/cancel/`, // POST
} as const;

// =====
// Reviews – отзывы
// =====
export const REVIEW = {
  LIST: '/reviews/', // GET/POST (?product=uuid)
  DETAIL: (id: number) => `/reviews/${id}/`, // GET/PATCH/DELETE
} as const;

// =====
// Discounts – купоны
// =====
export const DISCOUNT = {
  COUPONS: '/discounts/coupons/', // GET (staff)
  APPLY: '/discounts/apply/', // POST {order_id, code}
  REMOVE: '/discounts/remove/', // POST {order_id}
  PREVIEW: '/discounts/preview/', // POST {order_id, code}
} as const;

// =====
// Shipping – доставка
// =====
export const SHIPPING = {
  METHODS: '/shipping/methods/', // GET
  CALCULATE: '/shipping/calculate/', // POST {address_id, items}
}

```

```

SHIPMENTS: '/shipping/shipments/', // GET/POST
SHIPMENT_CREATE: '/shipping/shipments/create/', // POST (staff)
SHIPMENT: (id: number) => `/shipping/shipments/${id}/`, // GET
SHIPMENT_STATUS: (id: number) => `/shipping/shipments/${id}/status/`, // PATCH
SHIPMENT_TRACKING: (id: number) => `/shipping/shipments/${id}/tracking/`, // POST
TRACK: (tracking: string) => `/shipping/track/${tracking}/`, // GET (public)
} as const;

// =====
// Wishlist – избранное (требуется JWT)
// =====
export const WISHLIST = {
  DETAIL: '/wishlist/', // GET
  ADD: '/wishlist/add/', // POST {variant_id}
  REMOVE: (id: number) => `/wishlist/remove/${id}/`, // DELETE
  MOVE_TO_CART: '/wishlist/move-to-cart/', // POST {item_id}
  CLEAR: '/wishlist/clear/', // DELETE
} as const;

// =====
// Notifications – уведомления (требуется JWT)
// =====
export const NOTIFICATION = {
  LIST: '/notifications/', // GET
  UNREAD: '/notifications/unread/', // GET
  UNREAD_COUNT: '/notifications/unread-count/', // GET
  MARK_READ: (id: number) => `/notifications/${id}/read/`, // POST
  READ_ALL: '/notifications/read-all/', // POST
} as const;

// =====
// Analytics – аналитика (только admin)
// =====
export const ANALYTICS = {
  DASHBOARD: '/analytics/dashboard/',
  SALES: '/analytics/sales/',
  SALES_TIMELINE: '/analytics/sales/timeline/',
  TOP_PRODUCTS: '/analytics/top-products/',
  TOP_CATEGORIES: '/analytics/top-categories/',
  TOP_CUSTOMERS: '/analytics/top-customers/',
  CONVERSION: '/analytics/conversion/',
  MOST_VIEWED: '/analytics/most-viewed/',
} as const;

// =====
// Health – проверка бэкенда (публичный)
// =====
export const HEALTH = '/health/' as const;

```


4.5 Проверка: health-check из браузера

[\[DOC\] DRF API docs](#)

Давай убедимся, что API-клиент работает. Временно замени `src/App.tsx`:

```
// src/App.tsx – ВРЕМЕННАЯ проверка (потом заменим)

import { useEffect, useState } from 'react';
import api from './api/client';
import { HEALTH } from './api/endpoints';

function App() {
  const [health, setHealth] = useState<string>('Проверяю...');

  useEffect(() => {
    api.get(HEALTH)
      .then(res => setHealth(JSON.stringify(res.data)))
      .catch(err => setHealth(`Ошибка: ${err.message}`));
  }, []);

  return (
    <div style={{ padding: '40px', fontFamily: 'monospace' }}>
      <h1>[TEST] API Connection Test</h1>
      <p>Backend health: {health}</p>
    </div>
  );
}

export default App;
```

7 Запускаем оба сервера

```
# Терминал 1 – Django (бэкенд):
cd I:\NewPythonProjects\Amazone_Clone
python manage.py runserver
# → Starting development server at http://127.0.0.1:8000/

# Терминал 2 – React (фронтенд):
cd I:\NewPythonProjects\frontend
npm run dev
# → Local: http://localhost:5173/
```

Открой **http://localhost:5173**. Должно быть:

[TEST] API Connection Test

```
Backend health: {"status":"ok","version":"1.0.0","database":"ok"}
```

ДА Если видишь {"status":"ok"} – всё настроено правильно!

React подключился к Django через Vite проху, получил JSON и отобразил его.
Основная инфраструктура работает.

[!!] Если видишь «Ошибка: Network Error»

Проверь по порядку:

1. Django запущен? (`python manage.py runserver`)
2. Vite проху настроен? (vite.config.ts содержит `проху: { '/api': ... }`)
3. Перезапусти Vite после изменения vite.config.ts!
4. Открой `http://localhost:8000/api/v1/health/` прямо в браузере – должен вернуть JSON

Часть 5. Глобальное состояние (Zustand)

5.1 Что такое Zustand и зачем он нужен

[DOC] [Zustand GitHub](#)

В React у каждого компонента есть свой **локальное состояние** (`useState`). Но некоторые данные нужны **во многих местах**:

- **Пользователь** — нужен в хедере, на странице профиля, при оформлении заказа
- **Корзина** — нужна в хедере (количество товаров), на странице корзины, при добавлении товара
- **Уведомления** — нужны в хедере (колокольчик), на странице уведомлений

Zustand — это «глобальная переменная» для React. Любой компонент может прочитать или изменить данные.

Аналогия с Django

Django	React + Zustand
<code>request.user</code>	<code>useAuthStore(state => state.user)</code>
<code>request.session['cart']</code>	<code>useCartStore(state => state.cart)</code>
<code>login(request, user)</code>	<code>useAuthStore.getState().login(email, pwd)</code>
<code>logout(request)</code>	<code>useAuthStore.getState().logout()</code>

Почему Zustand, а не Redux?

Аспект	Redux	Zustand
Количество кода	Много (actions, reducers, selectors, store)	Мало (одна функция create)
Boilerplate	Высокий	Минимальный
Кривая обучения	Крутая	Пологая
Производительность	Хорошая	Отличная

5.2 authStore – пользователь, логин, регистрация

[DOC] [SimpleJWT](#)

8

Создай файл `src/stores/authStore.ts`

```

// src/stores/authStore.ts
// Глобальное хранилище пользователя.
// Zustand – простой state manager (замена Redux/Context).
//
// ЧТО ХРАНИТСЯ:
//   - user: объект пользователя (null если не авторизован)
//   - isLoading: загружается ли профиль (показываем спиннер)
//   - isAuthenticated: авторизован ли пользователь
//
// ДЕЙСТВИЯ:
//   - login(email, password) – вход, получение JWT
//   - register(data) – регистрация + автологин
//   - logout() – очистка токенов
//   - fetchUser() – получить профиль при загрузке приложения

import { create } from 'zustand';
import api from '../api/client';
import { AUTH, USER } from '../api/endpoints';

// — TypeScript-типы —

interface UserProfile {
  timezone: string;
  language: string;
  avatar: string | null;
}

interface User {
  id: number;
  email: string;
  username: string;
  first_name: string;
  last_name: string;
  phone: string;
  profile: UserProfile | null;
}

interface RegisterData {
  email: string;
  username: string;
  password: string;
  password_confirm: string;
  first_name?: string;
  last_name?: string;
}

interface AuthState {
  // — Состояние (данные) —
  user: User | null;
  isLoading: boolean;
  isAuthenticated: boolean;

  // — Действия (функции, меняющие состояние) —

```

```

login: (email: string, password: string) => Promise<void>;
register: (data: RegisterData) => Promise<void>;
logout: () => void;
fetchUser: () => Promise<void>;
}

// — Store —
// create() принимает функцию (set, get) => ({...state, ...actions})
// set() – обновляет состояние (как this.setState в классовых компонентах)
// get() – читает текущее состояние

export const useAuthStore = create<AuthState>((set, get) => ({
  // Начальное состояние:
  user: null,
  isLoading: true,          // true – при первой загрузке проверяем токен
  isAuthenticated: false,

  // — LOGIN —
  // Шаг 1: POST /auth/login/ → получаем {access, refresh}
  // Шаг 2: Сохраняем токены в localStorage
  // Шаг 3: GET /users/me/ → получаем профиль пользователя
  // Шаг 4: Обновляем состояние
  login: async (email, password) => {
    // ① Получаем JWT-токены
    const { data } = await api.post(AUTH.LOGIN, { email, password });

    // ② Сохраняем в localStorage (для интерцептора)
    localStorage.setItem('access_token', data.access);
    localStorage.setItem('refresh_token', data.refresh);

    // ③ Получаем профиль пользователя
    const { data: user } = await api.get(USER.ME);

    // ④ Обновляем глобальное состояние
    set({
      user,
      isAuthenticated: true,
      isLoading: false,
    });
  },

  // — REGISTER —
  // Регистрация + автоматический вход
  register: async (registerData) => {
    // ① Создаём аккаунт
    await api.post(AUTH.REGISTER, registerData);

    // ② Автоматически логинимся
    // get() – читаем текущее состояние (нужен для доступа к login)
    await get().login(registerData.email, registerData.password);
  },

  // — LOGOUT —

```

```

logout: () => {
  // Удаляем токены
  localStorage.removeItem('access_token');
  localStorage.removeItem('refresh_token');

  // Сбрасываем состояние
  set({
    user: null,
    isAuthenticated: false,
    isLoading: false,
  });
},

// — FETCH USER —
// Вызывается при первой загрузке приложения.
// Если в localStorage есть токен – проверяем, валидный ли он.
fetchUser: async () => {
  const token = localStorage.getItem('access_token');

  // Нет токена – не авторизован
  if (!token) {
    set({ isLoading: false });
    return;
  }

  try {
    // Пробуем получить профиль
    const { data } = await api.get(USER.ME);
    set({
      user: data,
      isAuthenticated: true,
      isLoading: false,
    });
  } catch {
    // Токен невалидный (истёк + refresh не помог) – чистим
    localStorage.removeItem('access_token');
    localStorage.removeItem('refresh_token');
    set({
      user: null,
      isAuthenticated: false,
      isLoading: false,
    });
  }
},
}));

```

Как использовать authStore в компоненте

```
// Пример: показываем имя пользователя в хедере
import { useAuthStore } from '../stores/authStore';

function Header() {
  // Подписываемся на user и isAuthenticated
  const { user, isAuthenticated, logout } = useAuthStore();

  return (
    <header>
      {isAuthenticated ? (
        <>
          <span>Привет, {user?.first_name || user?.username}!</span>
          <button onClick={logout}>Выйти</button>
        </>
      ) : (
        <a href="/login">Войти</a>
      )}
    </header>
  );
}
```

5.3 cartStore – корзина, добавление, удаление

9 Создай файл src/stores/cartStore.ts


```
// src/stores/cartStore.ts
// Глобальное хранилище корзины.
//
// ЧТО ХРАНИТСЯ:
// - cart: объект корзины с товарами (null = не загружена)
// - isCartOpen: открыта ли боковая панель корзины
// - isLoading: загружается ли
//
// ДЕЙСТВИЯ:
// - fetchCart() – получить корзину с сервера
// - addItem(variantId, quantity) – добавить товар
// - updateItem(itemId, quantity) – изменить количество
// - removeItem(itemId) – удалить позицию
// - clearCart() – очистить корзину
// - toggleCart() – открыть/закрыть боковую панель

import { create } from 'zustand';
import api from '../api/client';
import { CART } from '../api/endpoints';

// — Типы —

interface CartItem {
  id: number;
  variant_id: number;
  product_name: string;
  sku: string;
  unit_price: string;
  quantity: number;
  total_price: string;
  image: string | null;
}

interface Cart {
  id: number;
  items: CartItem[];
  total_quantity: number;
  total_price: string;
}

interface CartState {
  cart: Cart | null;
  isCartOpen: boolean;
  isLoading: boolean;

  fetchCart: () => Promise<void>;
  addItem: (variantId: number, quantity?: number) => Promise<void>;
  updateItem: (itemId: number, quantity: number) => Promise<void>;
  removeItem: (itemId: number) => Promise<void>;
  clearCart: () => Promise<void>;
  toggleCart: () => void;
}
```

```

export const useCartStore = create<CartState>((set, get) => ({
  cart: null,
  isCartOpen: false,
  isLoading: false,

  // — Получить корзину —
  fetchCart: async () => {
    try {
      const { data } = await api.get(CART.CART);
      set({ cart: data });
    } catch {
      // Гостевая корзина может не существовать — это нормально
    }
  },

  // — Добавить товар в корзину —
  // После добавления сервер возвращает обновлённую корзину
  addItem: async (variantId, quantity = 1) => {
    set({ isLoading: true });
    try {
      const { data } = await api.post(CART.ITEMS, {
        variant_id: variantId,
        quantity,
      });
      set({ cart: data, isLoading: false });
      // Автоматически открываем боковую панель корзины
      set({ isCartOpen: true });
    } catch (error) {
      set({ isLoading: false });
      throw error; // Пробрасываем ошибку для отображения в UI
    }
  },

  // — Изменить количество —
  updateItem: async (itemId, quantity) => {
    set({ isLoading: true });
    try {
      const { data } = await api.patch(CART.ITEM(itemId), { quantity });
      set({ cart: data, isLoading: false });
    } catch (error) {
      set({ isLoading: false });
      throw error;
    }
  },

  // — Удалить позицию —
  removeItem: async (itemId) => {
    set({ isLoading: true });
    try {
      const { data } = await api.delete(CART.ITEM(itemId));
      set({ cart: data, isLoading: false });
    } catch (error) {
      set({ isLoading: false });
    }
  }
});

```

```
        throw error;
      }
    },

    // — Очистить корзину —
    clearCart: async () => {
      const { data } = await api.delete(CART.CART);
      set({ cart: data });
    },

    // — Открыть/закрыть боковую панель —
    toggleCart: () => set((state) => ({ isCartOpen: !state.isCartOpen })),
  }));
```

5.4 wishlistStore – избранное

10 Создай файл `src/stores/wishlistStore.ts`

```
// src/stores/wishlistStore.ts
// Глобальное хранилище списка желаний (wishlist / избранное).
// Требуется JWT — только авторизованные пользователи.

import { create } from 'zustand';
import api from '../api/client';
import { WISHLIST } from '../api/endpoints';

interface WishlistItem {
  id: number;
  variant_id: number;
  product_name: string;
  added_at: string;
}

interface Wishlist {
  id: number;
  items: WishlistItem[];
  items_count: number;
}

interface WishlistState {
  wishlist: Wishlist | null;
  isLoading: boolean;

  fetchWishlist: () => Promise<void>;
  addItem: (variantId: number) => Promise<void>;
  removeItem: (itemId: number) => Promise<void>;
  moveToCart: (itemId: number) => Promise<void>;
  clearWishlist: () => Promise<void>;
}

export const useWishlistStore = create<WishlistState>((set) => ({
  wishlist: null,
  isLoading: false,

  fetchWishlist: async () => {
    try {
      const { data } = await api.get(WISHLIST.DETAIL);
      set({ wishlist: data });
    } catch {
      // Не авторизован — wishlist недоступен
    }
  },

  addItem: async (variantId) => {
    set({ isLoading: true });
    const { data } = await api.post(WISHLIST.ADD, { variant_id: variantId });
    set({ wishlist: data, isLoading: false });
  },

  removeItem: async (itemId) => {
    set({ isLoading: true });
  },
});
```

```
const { data } = await api.delete(WISHLIST.REMOVE(itemId));
set({ wishlist: data, isLoading: false });
},

moveToCart: async (itemId) => {
  set({ isLoading: true });
  const { data } = await api.post(WISHLIST.MOVE_TO_CART, { item_id: itemId });
  set({ wishlist: data, isLoading: false });
},

clearWishlist: async () => {
  const { data } = await api.delete(WISHLIST.CLEAR);
  set({ wishlist: data });
},
}));
```

5.5 notificationStore – уведомления

11 Создай файл `src/stores/notificationStore.ts`

```
// src/stores/notificationStore.ts
// Глобальное хранилище уведомлений.
// Отображается в хедере (колокольчик с бейджем).

import { create } from 'zustand';
import api from '../api/client';
import { NOTIFICATION } from '../api/endpoints';

interface Notification {
  id: number;
  type: string;           // order_created, payment_success, etc.
  title: string;
  message: string;
  is_read: boolean;
  created_at: string;
}

interface NotificationState {
  notifications: Notification[];
  unreadCount: number;
  isLoading: boolean;

  fetchNotifications: () => Promise<void>;
  fetchUnreadCount: () => Promise<void>;
  markRead: (id: number) => Promise<void>;
  markAllRead: () => Promise<void>;
}

export const useNotificationStore = create<NotificationState>((set) => ({
  notifications: [],
  unreadCount: 0,
  isLoading: false,

  fetchNotifications: async () => {
    set({ isLoading: true });
    const { data } = await api.get(NOTIFICATION.LIST);
    set({ notifications: data.results || data, isLoading: false });
  },

  fetchUnreadCount: async () => {
    const { data } = await api.get(NOTIFICATION.UNREAD_COUNT);
    set({ unreadCount: data.count || data.unread_count || 0 });
  },

  markRead: async (id) => {
    await api.post(NOTIFICATION.MARK_READ(id));
    // Обновляем локально (без нового запроса)
    set((state) => ({
      notifications: state.notifications.map(n =>
        n.id === id ? { ...n, is_read: true } : n
      ),
      unreadCount: Math.max(0, state.unreadCount - 1),
    }));
  }
}));
```

```

    },

    markAllRead: async () => {
      await api.post(NOTIFICATION.READ_ALL);
      set((state) => ({
        notifications: state.notifications.map(n => ({ ...n, is_read: true })),
        unreadCount: 0,
      }));
    },
  }));
}));

```

[!] Как работают stores вместе

Каждый store независим. Но они могут взаимодействовать. Например, при `moveToCart` в `wishlistStore`, нужно обновить `cartStore`:

```

// В wishlistStore:
moveToCart: async (itemId) => {
  const { data } = await api.post(WISHLIST.MOVE_TO_CART, { item_id: itemId });
  set({ wishlist: data });
  // Обновляем корзину!
  const { fetchCart } = useCartStore.getState();
  await fetchCart();
},

```

Часть 6. Роутинг (React Router)

React Router — это библиотека для навигации между страницами без перезагрузки. Аналог `urls.py` в Django.

Сравнение

Django (<code>urls.py</code>)	React (<code>router.tsx</code>)
<code>path('products/', ProductListView)</code>	<code>{ path: 'products', element: <ProductListPage /> }</code>
<code>path('products/<slug>/')</code>	<code>{ path: 'products/:slug', element: <ProductDetailPage /> }</code>
<code>login_required</code>	<code><ProtectedRoute></code>

6.1 Настройка маршрутов

[\[DOC\] React Router v6](#)

12 Создай файл `src/router.tsx`


```

// src/router.tsx
// Все маршруты приложения.
// React Router v6+ использует JSX для описания маршрутов.
//
// СТРУКТУРА:
//   / (Layout – хедер + футер на ВСЕХ страницах)
//   ├── / → HomePage
//   ├── /products → ProductListPage
//   ├── /products/:slug → ProductDetailPage
//   ├── /login → LoginPage
//   ├── /register → RegisterPage
//   └── (ProtectedRoute – только для авторизованных)
//       ├── /profile → ProfilePage
//       ├── /cart → CartPage
//       ├── /checkout → OrderCreatePage
//       ├── /orders → OrderListPage
//       ├── /orders/:orderNumber → OrderDetailPage
//       └── /wishlist → WishlistPage
//
// ProtectedRoute – компонент-обёртка, который проверяет isAuthenticated.
// Если не авторизован → перенаправляет на /login.

import { createBrowserRouter } from 'react-router-dom';
import { Layout } from '../components/Layout';
import { HomePage } from '../pages/HomePage';
import { LoginPage } from '../pages/LoginPage';
import { RegisterPage } from '../pages/RegisterPage';
import { ProductListPage } from '../pages/ProductListPage';
import { ProductDetailPage } from '../pages/ProductDetailPage';
import { CartPage } from '../pages/CartPage';
import { OrderCreatePage } from '../pages/OrderCreatePage';
import { OrderListPage } from '../pages/OrderListPage';
import { OrderDetailPage } from '../pages/OrderDetailPage';
import { ProfilePage } from '../pages/ProfilePage';
import { WishlistPage } from '../pages/WishlistPage';
import { ProtectedRoute } from '../components/ProtectedRoute';

export const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />, // Общий каркас (хедер + футер)
    children: [
      // == Публичные страницы ==
      { index: true, element: <HomePage /> },
      { path: 'products', element: <ProductListPage /> },
      { path: 'products/:slug', element: <ProductDetailPage /> },
      { path: 'login', element: <LoginPage /> },
      { path: 'register', element: <RegisterPage /> },

      // == Защищённые страницы (только для авторизованных) ==
      {
        element: <ProtectedRoute />, // Обёртка-проверка
        children: [

```

```

    { path: 'profile', element: <ProfilePage /> },
    { path: 'cart', element: <CartPage /> },
    { path: 'checkout', element: <OrderCreatePage /> },
    { path: 'orders', element: <OrderListPage /> },
    { path: 'orders/:orderNumber', element: <OrderDetailPage /> },
    { path: 'wishlist', element: <WishlistPage /> },
  ],
},
],
},
]);

```

Обнови src/main.tsx

```

// src/main.tsx
// Точка входа – подключаем RouterProvider

import React from 'react';
import ReactDOM from 'react-dom/client';
import { RouterProvider } from 'react-router-dom';
import { router } from './router';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>,
);

```

6.2 ProtectedRoute – защита страниц

[DOC] [React Router: Outlet](#)

13 Создай файл src/components/ProtectedRoute.tsx

```
// src/components/ProtectedRoute.tsx
// Компонент-обёртка для защищённых страниц.
// Если пользователь не авторизован → перенаправляем на /login.
//
// КАК РАБОТАЕТ:
//   <ProtectedRoute>
//     <ProfilePage />   ← отрендерится ТОЛЬКО если авторизован
//   </ProtectedRoute>
//
// Аналог в Django: @login_required декоратор

import { Navigate, Outlet } from 'react-router-dom';
import { useAuthStore } from '../stores/authStore';

export function ProtectedRoute() {
  const { isAuthenticated, isLoading } = useAuthStore();

  // Если профиль ещё загружается — показываем спиннер
  if (isLoading) {
    return (
      <div style={{
        display: 'flex',
        justifyContent: 'center',
        alignItems: 'center',
        height: '50vh'
      }}>
        <div style={{ fontSize: '24px' }}>... Загрузка...</div>
      </div>
    );
  }

  // Не авторизован → редирект на /login
  // replace — чтобы кнопка «Назад» не вернула на защищённую страницу
  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }

  // Авторизован → рендерим дочерние маршруты
  // Outlet — это «дырка», куда React Router вставит дочернюю страницу
  return <Outlet />;
}
```

[!] Outlet vs children

`<Outlet />` — это специальный компонент React Router, который показывает «дочерний» маршрут. Это не `children`. Когда URL = `/profile`, Outlet покажет `<ProfilePage />`. Когда URL = `/cart`, Outlet покажет `<CartPage />`.

6.3 Layout – общий каркас страниц

14 Создай файл `src/components/Layout.tsx`

```
// src/components/Layout.tsx
// Общий каркас всех страниц: хедер + основной контент + футер.
//
// СТРУКТУРА:
//
// | HEADER (логотип, поиск, навигация) |
// |                                     |
// | <Outlet /> – контент текущей       |
// | страницы (Home, Catalog, Cart...) |
// |                                     |
// | FOOTER (копирайт, ссылки)         |
// |
//
// Layout рендерится на КАЖДОЙ странице (благодаря router.tsx).

import { useEffect } from 'react';
import { Outlet, Link, useNavigate } from 'react-router-dom';
import { useAuthStore } from '../stores/authStore';
import { useCartStore } from '../stores/cartStore';
import { useNotificationStore } from '../stores/notificationStore';

export function Layout() {
  const { user, isAuthenticated, logout, fetchUser } = useAuthStore();
  const { cart, isCartOpen, toggleCart, fetchCart, removeItem, updateItem } = useCartStore();
  const { unreadCount, fetchUnreadCount } = useNotificationStore();
  const navigate = useNavigate();

  // При первой загрузке – получаем пользователя, корзину и уведомления
  useEffect(() => {
    fetchUser();
    fetchCart();
    fetchUnreadCount();
  }, []);

  const handleLogout = () => {
    logout();
    navigate('/');
  };

  return (
    <div style={{ minHeight: '100vh', display: 'flex', flexDirection: 'column' }}>

      { /* == ХЕДЕР == */ }
      <header style={{
        background: '#131921',
        color: 'white',
        padding: '10px 20px',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'space-between',
        position: 'sticky',

```

```

    top: 0,
    zIndex: 100,
  }}>

  {/* Логотип */}
  <Link to="/" style={{
    color: 'white', textDecoration: 'none',
    fontSize: '24px', fontWeight: 'bold'
  }}>
    [CART] Amazone
  </Link>

  {/* Поиск */}
  <input
    type="text"
    placeholder="Искать товары..."
    style={{
      flex: 1, margin: '0 20px', padding: '8px 12px',
      borderRadius: '4px', border: 'none', maxWidth: '600px'
    }}
  />

  {/* Навигация */}
  <div style={{ display: 'flex', gap: '16px', alignItems: 'center' }}>
    <Link to="/products" style={{ color: 'white', textDecoration: 'none' }}>
      Каталог
    </Link>

    {isAuthenticated ? (
      <>
        <Link to="/wishlist" style={{ color: 'white', textDecoration: 'none' }}>
          ♥ Избранное
        </Link>
        <Link to="/orders" style={{ color: 'white', textDecoration: 'none' }}>
          [BOX] Заказы
        </Link>
        <Link to="/notifications" style={{ color: 'white', textDecoration: 'none', position: 'relative' }}>
          [BELL]
          {unreadCount > 0 && (
            <span style={{
              position: 'absolute', top: -8, right: -8,
              background: '#f44336', color: 'white',
              borderRadius: '50%', width: 18, height: 18,
              fontSize: '11px', display: 'flex',
              alignItems: 'center', justifyContent: 'center',
            }}>{unreadCount}</span>
          )}
        </Link>
        <Link to="/profile" style={{ color: 'white', textDecoration: 'none' }}>
          [USER] {user?.first_name || user?.username}
        </Link>
        <button
          onClick={handleLogout}

```

```

        style={{ background: 'none', border: 'none', color: '#ffa500', cursor: 'pointer'
      >
        Выйти
      </button>
    </>
  ) : (
    <Link to="/login" style={{ color: 'white', textDecoration: 'none' }}>
      Войти
    </Link>
  )}

  {/* Кнопка корзины */}
  <button
    onClick={toggleCart}
    style={{
      background: 'none', border: 'none', color: 'white',
      cursor: 'pointer', fontSize: '16px', position: 'relative'
    }}
  >
    [CART]
    {cart && cart.total_quantity > 0 && (
      <span style={{
        position: 'absolute', top: -8, right: -12,
        background: '#ffa500', color: '#131921',
        borderRadius: '50%', width: 20, height: 20,
        fontSize: '11px', fontWeight: 'bold',
        display: 'flex', alignItems: 'center', justifyContent: 'center',
      }}>{cart.total_quantity}</span>
    )}
  </button>
</div>
</header>

{/* == БОКОВАЯ ПАНЕЛЬ КОРЗИНЫ == */}
{isCartOpen && (
  <div style={{
    position: 'fixed', top: 0, right: 0, bottom: 0,
    width: '400px', background: 'white',
    boxShadow: '-2px 0 10px rgba(0,0,0,0.2)',
    zIndex: 200, padding: '20px',
    overflowY: 'auto',
  }}>
    <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '20px'
      <h2>[CART] Корзина</h2>
      <button onClick={toggleCart} style={{ background: 'none', border: 'none', fontSize:
    </div>

    {cart && cart.items && cart.items.length > 0 ? (
      <>
        {cart.items.map(item => (
          <div key={item.id} style={{
            display: 'flex', gap: '12px', padding: '12px 0',
            borderBottom: '1px solid #eee'

```

```

    }}>
    <div style={{ flex: 1 }}>
      <div style={{ fontWeight: 'bold', fontSize: '14px' }}>{item.product_name}</div>
      <div style={{ color: '#888', fontSize: '12px' }}>{item.sku}</div>
      <div style={{ display: 'flex', alignItems: 'center', gap: '8px', marginTop: '8px' }}>
        <button
          onClick={() => updateItem(item.id, Math.max(1, item.quantity - 1))}
          style={{ width: 28, height: 28, border: '1px solid #ddd', borderRadius: 4 }}>-</button>
        <span>{item.quantity}</span>
        <button
          onClick={() => updateItem(item.id, item.quantity + 1)}
          style={{ width: 28, height: 28, border: '1px solid #ddd', borderRadius: 4 }}>+</button>
      </div>
    </div>
    <div style={{ textAlign: 'right' }}>
      <div style={{ color: '#c00', fontWeight: 'bold' }}>{item.total_price} ₽</div>
      <button
        onClick={() => removeItem(item.id)}
        style={{ background: 'none', border: 'none', color: 'red', cursor: 'pointer' }}>
        >Удалить</button>
      </div>
    </div>
  ))}
  <div style={{
    marginTop: '16px', padding: '16px 0',
    borderTop: '2px solid #131921',
    display: 'flex', justifyContent: 'space-between',
    fontSize: '18px', fontWeight: 'bold'
  }}>
    <span>Итого:</span>
    <span style={{ color: '#c00' }}>{cart.total_price} ₽</span>
  </div>
  <Link
    to="/cart"
    onClick={toggleCart}
    style={{
      display: 'block', textAlign: 'center', padding: '12px',
      background: '#ffa500', color: '#131921',
      textDecoration: 'none', borderRadius: '4px', fontWeight: 'bold',
    }}>
    > Перейти в корзину
  </Link>
</>
) : (
  <p style={{ color: '#888', textAlign: 'center' }}>Корзина пуста</p>
)
</div>
)}

{/* Затемнение фона при открытой корзине */}

```



```

{isCartOpen && (
  <div
    onClick={toggleCart}
    style={{
      position: 'fixed', top: 0, left: 0, right: 0, bottom: 0,
      background: 'rgba(0,0,0,0.5)', zIndex: 150,
    }}
  />
)}

{/* === ОСНОВНОЙ КОНТЕНТ === */}
<main style={{ flex: 1, padding: '20px', maxWidth: '1200px', margin: '0 auto', width: '100%' }}>
  <Outlet /> {/* ← Сюда React Router вставляет текущую страницу */}
</main>

{/* === ФУТЕР === */}
<footer style={{
  background: '#131921',
  color: '#aaa',
  padding: '20px',
  textAlign: 'center',
  fontSize: '14px',
}}>
  <p>© 2026 Amazone Clone. Учебный проект на Django + React.</p>
  <div style={{ marginTop: '8px', display: 'flex', gap: '16px', justifyContent: 'center' }}>
    <Link to="/products" style={{ color: '#ffa500', textDecoration: 'none' }}>Каталог</Link>
    <a href="http://localhost:8000/api/v1/docs/" target="_blank" style={{ color: '#ffa500' }}>API Docs</a>
    <a href="http://localhost:8000/admin/" target="_blank" style={{ color: '#ffa500' }}>Админ</a>
  </div>
</footer>
</div>
);
}

```

Часть 7. Страницы авторизации

7.1 LoginPage – вход по email

[DOC] [SimpleJWT Login](#)

15 Создай файл `src/pages/LoginPage.tsx`

```
// src/pages/LoginPage.tsx
// Страница входа по email + пароль.
//
// ПОТОК ДАННЫХ:
// 1. Пользователь вводит email и пароль
// 2. Нажимает «Войти»
// 3. Вызываем authStore.login(email, password)
// 4. Внутри login: POST /auth/login/ → получаем {access, refresh}
// 5. Внутри login: GET /users/me/ → получаем профиль
// 6. Перенаправляем на главную страницу
//
// ОБРАБОТКА ОШИБОК:
// - 401: неверный email или пароль → показываем «Неверный логин или пароль»
// - Network Error: бэкенд недоступен → показываем «Сервер недоступен»
// - Любая другая ошибка → показываем сообщение

import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuthStore } from '../stores/authStore';

export function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  const login = useAuthStore(state => state.login);
  const navigate = useNavigate();

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError('');
    setIsSubmitting(true);

    try {
      await login(email, password);
      // Успешный вход → на главную
      navigate('/', { replace: true });
    } catch (err: any) {
      // Обрабатываем ошибку
      if (err.response?.status === 401) {
        setError('Неверный email или пароль');
      } else if (err.message === 'Network Error') {
        setError('Сервер недоступен. Проверьте, запущен ли бэкенд.');
```

```

<div style={{
  maxWidth: '400px', margin: '40px auto',
  padding: '32px',
  border: '1px solid #ddd', borderRadius: '8px'
}}>
  <h1 style={{ textAlign: 'center', marginBottom: '24px' }}>Вход</h1>

  {error && (
    <div style={{
      background: '#fce4ec', color: '#c62828',
      padding: '12px', borderRadius: '4px', marginBottom: '16px'
    }}>
      {error}
    </div>
  )}

  <form onSubmit={handleSubmit}>
    <div style={{ marginBottom: '16px' }}>
      <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>
        Email
      </label>
      <input
        type="email"
        value={email}
        onChange={e => setEmail(e.target.value)}
        required
        placeholder="user@mail.com"
        style={{
          width: '100%', padding: '10px',
          border: '1px solid #ccc', borderRadius: '4px',
          boxSizing: 'border-box'
        }}
      />
    </div>

    <div style={{ marginBottom: '16px' }}>
      <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>
        Пароль
      </label>
      <input
        type="password"
        value={password}
        onChange={e => setPassword(e.target.value)}
        required
        placeholder="Введите пароль"
        style={{
          width: '100%', padding: '10px',
          border: '1px solid #ccc', borderRadius: '4px',
          boxSizing: 'border-box'
        }}
      />
    </div>
  </form>

```

```

    <button
      type="submit"
      disabled={isSubmitting}
      style={{
        width: '100%', padding: '12px',
        background: '#ffa500', color: '#131921',
        border: 'none', borderRadius: '4px',
        fontSize: '16px', fontWeight: 'bold',
        cursor: isSubmitting ? 'not-allowed' : 'pointer',
        opacity: isSubmitting ? 0.7 : 1,
      }}
    >
      {isSubmitting ? 'Входим...' : 'Войти'}
    </button>
  </form>

  <p style={{ textAlign: 'center', marginTop: '16px', color: '#666' }}>
    Нет аккаунта?{' '}
    <Link to="/register" style={{ color: '#ffa500', fontWeight: 'bold' }}>
      Зарегистрироваться
    </Link>
  </p>
</div>
);
}

```

7.2 RegisterPage – регистрация

16 Создай файл `src/pages/RegisterPage.tsx`

```

// src/pages/RegisterPage.tsx
// Страница регистрации нового пользователя.
//
// ВАЛИДАЦИЯ:
//   - Все поля обязательны
//   - password_confirm должно совпадать с password
//   - Email должен быть уникальным (проверяет бэкенд)
//
// ПОСЛЕ УСПЕШНОЙ РЕГИСТРАЦИИ:
//   authStore.register() автоматически вызывает login()
//   → пользователь сразу заходит без дополнительного ввода

import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuthStore } from '../stores/authStore';

export function RegisterPage() {
  const [formData, setFormData] = useState({
    email: '',
    username: '',
    password: '',
    password_confirm: '',
    first_name: '',
    last_name: '',
  });
  const [error, setError] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);

  const register = useAuthStore(state => state.register);
  const navigate = useNavigate();

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setFormData(prev => ({ ...prev, [e.target.name]: e.target.value }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError('');

    // Клиентская валидация
    if (formData.password !== formData.password_confirm) {
      setError('Пароли не совпадают');
      return;
    }

    if (formData.password.length < 8) {
      setError('Пароль должен быть не менее 8 символов');
      return;
    }

    setIsSubmitting(true);

    try {

```

```

    await register(formData);
    navigate('/', { replace: true });
  } catch (err: any) {
    if (err.response?.data) {
      // Django возвращает ошибки валидации как объект:
      // {"email": ["Пользователь с таким email уже существует."],
      //  "username": ["Это поле обязательно."]}
      const messages = Object.values(err.response.data).flat();
      setError(messages.join(' '));
    } else {
      setError('Произошла ошибка. Попробуйте позже.');
```

```

    }
  } finally {
    setIsSubmitting(false);
  }
};
```

```

return (
```

```

  <div style={{
    maxWidth: '400px', margin: '40px auto',
    padding: '32px',
    border: '1px solid #ddd', borderRadius: '8px'
```

```

  }}>
  <h1 style={{ textAlign: 'center', marginBottom: '24px' }}>Регистрация</h1>
```

```

  {error && (
    <div style={{
      background: '#fce4ec', color: '#c62828',
      padding: '12px', borderRadius: '4px', marginBottom: '16px'
    }}>
      {error}
    </div>
  )}
```

```

  <form onSubmit={handleSubmit}>
    <div style={{ marginBottom: '12px' }}>
      <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Email *</label>
      <input type="email" name="email" value={formData.email} onChange={handleChange}
        required style={{ width: '100%', padding: '10px', border: '1px solid #ccc', borderRadius: '4px' }} />
    </div>
```

```

    <div style={{ marginBottom: '12px' }}>
      <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Имя пользователя *</label>
      <input type="text" name="username" value={formData.username} onChange={handleChange}
        required style={{ width: '100%', padding: '10px', border: '1px solid #ccc', borderRadius: '4px' }} />
    </div>
```

```

    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '12px', marginBottom: '12px' }}>
      <div>
        <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Имя *</label>
        <input type="text" name="first_name" value={formData.first_name} onChange={handleChange}
          style={{ width: '100%', padding: '10px', border: '1px solid #ccc', borderRadius: '4px' }} />
      </div>
```

```

    <div>
      <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Фамилия</label>
      <input type="text" name="last_name" value={formData.last_name} onChange={handleChange}
        style={{ width: '100%', padding: '10px', border: '1px solid #ccc', borderRadius: '4px' }} />
    </div>
  </div>

  <div style={{ marginBottom: '12px' }}>
    <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Пароль</label>
    <input type="password" name="password" value={formData.password} onChange={handleChange}
      required minLength={8} style={{ width: '100%', padding: '10px', border: '1px solid #ccc', borderRadius: '4px' }} />
  </div>

  <div style={{ marginBottom: '16px' }}>
    <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Подтверждение пароля</label>
    <input type="password" name="password_confirm" value={formData.password_confirm} onChange={handleChange}
      required style={{ width: '100%', padding: '10px', border: '1px solid #ccc', borderRadius: '4px' }} />
  </div>

  <button type="submit" disabled={isSubmitting} style={{
    width: '100%', padding: '12px',
    background: '#ffa500', color: '#131921',
    border: 'none', borderRadius: '4px',
    fontSize: '16px', fontWeight: 'bold',
    cursor: isSubmitting ? 'not-allowed' : 'pointer',
    opacity: isSubmitting ? 0.7 : 1,
  }}>
    {isSubmitting ? 'Регистрируем...' : 'Зарегистрироваться'}
  </button>
</form>

<p style={{ textAlign: 'center', marginTop: '16px', color: '#666' }}>
  Уже есть аккаунт?{' '}
  <Link to="/login" style={{ color: '#ffa500', fontWeight: 'bold' }}>Войти</Link>
</p>
</div>
);
}

```

7.3 Автоход при обновлении страницы

Когда пользователь обновляет страницу, React «забывает» всё состояние (включая `user` из `authStore`). Но JWT-токены всё ещё лежат в `localStorage`!

В `Layout.tsx` мы уже вызываем `fetchUser()` при загрузке. Давай разберём, как это работает:

Пользователь обновляет страницу (F5)



React перезагружается. `authStore: user=null, isAuthenticated=false`



`Layout.tsx` → `useEffect` → `fetchUser()`



`fetchUser()` проверяет `localStorage`:

- `access_token` есть? → ДА



GET `/api/v1/users/me/`

Authorization: Bearer `eyJhbG...`старый_access



Два варианта:

А) `access` ещё валиден (15 мин не прошло)

→ Django возвращает профиль

→ `isAuthenticated = true` ДА

Б) `access` истёк (15 мин прошло)

→ Django возвращает 401

→ Response interceptor: POST `/auth/refresh/`

→ Получаем новый `access`

→ Повторяем GET `/users/me/` с новым токеном

→ `isAuthenticated = true` ДА

⚠ Важный момент

В первые миллисекунды после загрузки `isAuthenticated = false`, а `isLoading = true`. Поэтому `ProtectedRoute` показывает спиннер, а не перенаправляет на `/login`. Когда `fetchUser` завершается, `isLoading` становится `false`, и страница показывается.

Часть 8. Каталог товаров

8.1 ProductListPage – список с фильтрами и поиском

[DOC] [DRF Pagination](#) · [DRF Filtering](#)

17 Создай файл `src/pages/ProductListPage.tsx`

```
// src/pages/ProductListPage.tsx
// Страница каталога товаров с поиском, фильтрами и пагинацией.
//
// КАК РАБОТАЕТ:
// 1. При загрузке: GET /api/v1/catalog/products/
// 2. Пользователь вводит поиск → обновляем URL-параметры → новый запрос
// 3. Пользователь выбирает фильтр → новый запрос
// 4. Пользователь кликает «Далее» → GET ...?page=2
//
// ДАННЫЕ ОТ DJANGO:
// {
//   "count": 156,
//   "next": "...?page=2",
//   "previous": null,
//   "results": [...массив товаров...]
// }

import { useEffect, useState } from 'react';
import { Link, useSearchParams } from 'react-router-dom';
import api from '../api/client';
import { CATALOG } from '../api/endpoints';
import { useCartStore } from '../stores/cartStore';

interface Product {
  id: string;
  name: string;
  slug: string;
  min_price: string;
  max_price: string;
  rating: string;
  reviews_count: number;
  views_count: number;
  main_image: string | null;
  brand: { name: string; slug: string } | null;
  category: { name: string; slug: string } | null;
}

interface ProductListResponse {
  count: number;
  next: string | null;
  previous: string | null;
  results: Product[];
}

export function ProductListPage() {
  // useSearchParams – хук для работы с URL-параметрами (?search=iphone&page=2)
  const [searchParams, setSearchParams] = useSearchParams();

  const [data, setData] = useState<ProductListResponse | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [searchQuery, setSearchQuery] = useState(searchParams.get('search') || '');

  const addItem = useCartStore(state => state.addItem);
```

```

// Читаем текущие параметры из URL
const currentPage = Number(searchParams.get('page')) || 1;
const currentSearch = searchParams.get('search') || '';
const currentCategory = searchParams.get('category') || '';
const currentBrand = searchParams.get('brand') || '';
const currentOrdering = searchParams.get('ordering') || '';

// Загружаем товары при изменении параметров
useEffect(() => {
  setIsLoading(true);

  // Строим параметры запроса
  const params: Record<string, string> = { page: String(currentPage) };
  if (currentSearch) params.search = currentSearch;
  if (currentCategory) params.category = currentCategory;
  if (currentBrand) params.brand = currentBrand;
  if (currentOrdering) params.ordering = currentOrdering;

  api.get(CATALOG.PRODUCTS, { params })
    .then(({ data }) => setData(data))
    .catch(() => setData(null))
    .finally(() => setIsLoading(false));
}, [currentPage, currentSearch, currentCategory, currentBrand, currentOrdering]);

// Обработчик поиска
const handleSearch = (e: React.FormEvent) => {
  e.preventDefault();
  const newParams = new URLSearchParams(searchParams);
  if (searchQuery) {
    newParams.set('search', searchQuery);
  } else {
    newParams.delete('search');
  }
  newParams.delete('page'); // Сбрасываем пагинацию
  setSearchParams(newParams);
};

// Обработчик сортировки
const handleSort = (ordering: string) => {
  const newParams = new URLSearchParams(searchParams);
  newParams.set('ordering', ordering);
  newParams.delete('page');
  setSearchParams(newParams);
};

// Переключение страницы
const goToPage = (page: number) => {
  const newParams = new URLSearchParams(searchParams);
  newParams.set('page', String(page));
  setSearchParams(newParams);
  window.scrollTo(0, 0); // Скроллим вверх
};

```

```

// Добавить в корзину
const handleAddToCart = async (variantId: number) => {
  try {
    await addItem(variantId);
  } catch {
    alert('Ошибка при добавлении в корзину');
  }
};

return (
  <div>
    <h1 style={{ marginBottom: '20px' }}>[BOX] Каталог товаров</h1>

    {/== Панель поиска и фильтров == */}
    <div style={{
      display: 'flex', gap: '12px', marginBottom: '20px',
      flexWrap: 'wrap', alignItems: 'center'
    }}>
      {/== Поиск */}
      <form onSubmit={handleSearch} style={{ display: 'flex', flex: 1, minWidth: '250px' }}>
        <input
          type="text"
          value={searchQuery}
          onChange={e => setSearchQuery(e.target.value)}
          placeholder="Поиск товаров..."
          style={{
            flex: 1, padding: '10px',
            border: '1px solid #ccc', borderRadius: '4px 0 0 4px'
          }}
        />
        <button type="submit" style={{
          padding: '10px 20px', background: '#ffa500', border: 'none',
          borderRadius: '0 4px 4px 0', cursor: 'pointer', fontWeight: 'bold',
        }}>
          [SEARCH]
        </button>
      </form>

      {/== Сортировка */}
      <select
        value={currentOrdering}
        onChange={e => handleSort(e.target.value)}
        style={{ padding: '10px', border: '1px solid #ccc', borderRadius: '4px' }}
      >
        <option value="">Сортировка</option>
        <option value="price">Цена: по возрастанию</option>
        <option value="-price">Цена: по убыванию</option>
        <option value="-rating">По рейтингу</option>
        <option value="name">По названию</option>
        <option value="-created_at">Новинки</option>
      </select>
    </div>
  )

```

```

{ /* Активные фильтры */ }
{ currentSearch && (
  <div style={{ marginBottom: '16px', color: '#666' }}>
    Поиск: «{currentSearch}»
    <button
      onClick={() => { setSearchQuery(''); setSearchParams(new URLSearchParams()); }}
      style={{ marginLeft: '8px', background: 'none', border: '1px solid #ccc', borderRa
    >× Сбросить</button>
  </div>
)}

{ /* == Загрузка == */ }
{ !isLoading && <div style={{ textAlign: 'center', padding: '40px' }}>... Загрузка...</div>

{ /* == Товары == */ }
{ !isLoading && data && (
  <>
    <div style={{ color: '#666', marginBottom: '12px' }}>
      Найдено: {data.count} товаров
    </div>

    <div style={{
      display: 'grid',
      gridTemplateColumns: 'repeat(auto-fill, minmax(240px, 1fr))',
      gap: '16px',
    }}>
      {data.results.map(product => (
        <div key={product.id} style={{
          border: '1px solid #ddd', borderRadius: '8px',
          padding: '16px', background: 'white',
          transition: 'box-shadow 0.2s',
        }}>
          { /* Картинка */ }
          <Link to={`\products/${product.slug}`}>
            <div style={{
              height: '180px', background: '#f5f5f5', borderRadius: '4px',
              display: 'flex', alignItems: 'center', justifyContent: 'center',
              marginBottom: '12px',
            }}>
              {product.main_image ? (
                <img src={product.main_image} alt={product.name}
                  style={{ maxWidth: '100%', maxHeight: '100%', objectFit: 'contain' }} /
              ) : (
                <span style={{ fontSize: '48px' }}>[BOX]</span>
              )}
            </div>
          </Link>

          { /* Информация */ }
          <Link to={`\products/${product.slug}`}
            style={{ textDecoration: 'none', color: 'inherit' }}>
            <div style={{ fontWeight: 'bold', fontSize: '14px', marginBottom: '4px' }}>

```

```

        {product.name}
      </div>
    </Link>

    {product.brand && (
      <div style={{ color: '#888', fontSize: '12px' }}>{product.brand.name}</div>
    )}

    <div style={{ color: '#c00', fontWeight: 'bold', fontSize: '18px', marginTop: '10px' }}>
      {product.min_price} ₽
    </div>

    <div style={{ fontSize: '12px', color: '#666', marginTop: '4px' }}>
      ★ {parseFloat(product.rating).toFixed(1)} ({product.reviews_count} отзывов)
    </div>
  </div>
)}}
```

```

{ /* == Пагинация == */
{data.count > 20 && (
  <div style={{
    display: 'flex', justifyContent: 'center',
    gap: '8px', marginTop: '24px'
  }}>
    <button
      disabled={!data.previous}
      onClick={() => goToPage(currentPage - 1)}
      style={{
        padding: '8px 16px', border: '1px solid #ddd',
        borderRadius: '4px', cursor: data.previous ? 'pointer' : 'not-allowed',
        opacity: data.previous ? 1 : 0.5,
      }}
    >← Назад</button>

    <span style={{ padding: '8px 16px' }}>
      Стр. {currentPage} из {Math.ceil(data.count / 20)}
    </span>

    <button
      disabled={!data.next}
      onClick={() => goToPage(currentPage + 1)}
      style={{
        padding: '8px 16px', border: '1px solid #ddd',
        borderRadius: '4px', cursor: data.next ? 'pointer' : 'not-allowed',
        opacity: data.next ? 1 : 0.5,
      }}
    >Далее →</button>
  </div>
)}}
```

```

</>
)}
```

```
    { /* == Пусто == */  
    {!isLoading && data && data.results.length === 0 && (  
      <div style={{ textAlign: 'center', padding: '40px', color: '#888' }}>  
        ☹ Товары не найдены  
      </div>  
    )}  
  </div>  
);  
}
```

8.2 ProductDetailPage – карточка товара

18 Создай файл `src/pages/ProductDetailPage.tsx`


```
// src/pages/ProductDetailPage.tsx
// Страница карточки товара: фото, описание, цена, варианты, отзывы.
//
// URL: /products/:slug (slug = "iphone-15-pro")
// useParams() достаёт slug из URL.
//
// ЗАПРОСЫ К БЭКЕНДУ:
// 1. GET /api/v1/catalog/products/{slug}/ – данные товара
// 2. GET /api/v1/reviews/?product={uuid} – отзывы

import { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';
import api from '../api/client';
import { CATALOG, REVIEW } from '../api/endpoints';
import { useCartStore } from '../stores/cartStore';

interface Variant {
  id: number;
  name: string;
  sku: string;
  price: string;
  sale_price: string | null;
  effective_price: string;
  is_available: boolean;
  attributes: Record<string, string>;
}

interface ProductDetail {
  id: string;
  name: string;
  slug: string;
  description: string;
  min_price: string;
  max_price: string;
  rating: string;
  reviews_count: number;
  views_count: number;
  main_image: string | null;
  images: Array<{ id: number; image: string; alt: string; is_main: boolean }>;
  brand: { name: string; slug: string } | null;
  category: { name: string; slug: string } | null;
  variants: Variant[];
  tags: Array<{ name: string; slug: string }>;
}

interface ReviewItem {
  id: number;
  user: { username: string; first_name: string };
  rating: number;
  text: string;
  verified_purchase: boolean;
  helpful_yes: number;
  helpful_no: number;
}
```

```

    created_at: string;
  }

export function ProductDetailPage() {
  // useParams() – хук для чтения динамических параметров URL
  // Если URL = /products/iphone-15-pro, то slug = "iphone-15-pro"
  const { slug } = useParams<{ slug: string }>();

  const [product, setProduct] = useState<ProductDetail | null>(null);
  const [reviews, setReviews] = useState<ReviewItem[]>([]);
  const [selectedVariant, setSelectedVariant] = useState<Variant | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [quantity, setQuantity] = useState(1);

  const addItem = useCartStore(state => state.addItem);

  useEffect(() => {
    if (!slug) return;
    setIsLoading(true);

    // Загружаем товар и отзывы параллельно
    Promise.all([
      api.get(CATALOG.PRODUCT(slug)),
      api.get(REVIEW.LIST, { params: { product: slug } }),
    ])
      .then(([productRes, reviewsRes]) => {
        setProduct(productRes.data);
        setReviews(reviewsRes.data.results || reviewsRes.data || []);

        // Выбираем первый доступный вариант
        const firstAvailable = productRes.data.variants?.find(
          (v: Variant) => v.is_available
        );
        if (firstAvailable) setSelectedVariant(firstAvailable);
      })
      .catch(() => setProduct(null))
      .finally(() => setIsLoading(false));
  }, [slug]);

  const handleAddToCart = async () => {
    if (!selectedVariant) return;
    try {
      await addItem(selectedVariant.id, quantity);
    } catch {
      alert('Ошибка при добавлении в корзину');
    }
  };

  if (isLoading) return <div style={{ textAlign: 'center', padding: '40px' }}>... Загрузка...</div>;
  if (!product) return <div style={{ textAlign: 'center', padding: '40px' }}>Товар не найден</div>;

  return (
    <div>

```

```

    { /* Хлебные крошки */ }
    <div style={{ color: '#666', fontSize: '14px', marginBottom: '20px' }}>
      <Link to="/">Главная</Link> → { ' ' }
      <Link to="/products">Каталог</Link> → { ' ' }
      {product.category && <><Link to={`/products?category=${product.category.slug}`}>{product
        {product.name}
      </div>

    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '32px' }}>
      { /* == ФОТО == */ }
      <div>
        <div style={{
          height: '400px', background: '#f5f5f5', borderRadius: '8px',
          display: 'flex', alignItems: 'center', justifyContent: 'center',
        }}>
          {product.main_image ? (
            <img src={product.main_image} alt={product.name}
              style={{ maxWidth: '100%', maxHeight: '100%', objectFit: 'contain' }} />
          ) : (
            <span style={{ fontSize: '80px' }}>[BOX]</span>
          )}
        </div>
        { /* Миниатюры */ }
        <div style={{ display: 'flex', gap: '8px', marginTop: '8px' }}>
          {product.images?.map(img => (
            <img key={img.id} src={img.image} alt={img.alt}
              style={{ width: 60, height: 60, objectFit: 'cover', borderRadius: '4px', border
            ))}
        </div>
      </div>

    { /* == ИНФОРМАЦИЯ == */ }
    <div>
      {product.brand && (
        <div style={{ color: '#888', marginBottom: '8px' }}>{product.brand.name}</div>
      )}
      <h1 style={{ fontSize: '24px', marginBottom: '12px' }}>{product.name}</h1>

      { /* Рейтинг */ }
      <div style={{ display: 'flex', alignItems: 'center', gap: '8px', marginBottom: '16px' }}>
        <span style={{ color: '#ffa500', fontSize: '18px' }}>{'★'.repeat(Math.round(parseFloat
        <span>{parseFloat(product.rating).toFixed(1)}</span>
        <span style={{ color: '#888' }}>({product.reviews_count} отзывов)</span>
        <span style={{ color: '#888' }}>[EYE] {product.views_count} просмотров</span>
      </div>

      { /* Цена */ }
      <div style={{ fontSize: '28px', fontWeight: 'bold', color: '#c00', marginBottom: '16px' }}>
        {selectedVariant?.effective_price || product.min_price} ₽
      </div>

      { /* Варианты */ }
      {product.variants && product.variants.length > 1 && (

```

```

<div style={{ marginBottom: '16px' }}>
  <h3>Варианты:</h3>
  <div style={{ display: 'flex', gap: '8px', flexWrap: 'wrap' }}>
    {product.variants.map(variant => (
      <button
        key={variant.id}
        onClick={() => setSelectedVariant(variant)}
        disabled={!variant.is_available}
        style={{
          padding: '8px 16px',
          border: selectedVariant?.id === variant.id ? '2px solid #ffa500' : '1px solid #ddd',
          borderRadius: '4px',
          cursor: variant.is_available ? 'pointer' : 'not-allowed',
          background: selectedVariant?.id === variant.id ? '#fff3e0' : 'white',
          opacity: variant.is_available ? 1 : 0.5,
        }}
      >
        {variant.name} – {variant.effective_price} ₽
      </button>
    ))}
  </div>
</div>
)}

{/* Количество + В корзину */}
<div style={{ display: 'flex', gap: '12px', alignItems: 'center', marginBottom: '24px' }}>
  <div style={{ display: 'flex', alignItems: 'center', border: '1px solid #ddd', padding: '5px' }}>
    <button onClick={() => setQuantity(Math.max(1, quantity - 1))}
      style={{ width: 36, height: 36, border: 'none', background: '#f5f5f5', cursor: 'pointer' }}>
    <span style={{ width: 40, textAlign: 'center', lineHeight: '36px' }}>{quantity}</span>
    <button onClick={() => setQuantity(quantity + 1)}
      style={{ width: 36, height: 36, border: 'none', background: '#f5f5f5', cursor: 'pointer' }}>
  </div>
  <button
    onClick={handleAddToCart}
    disabled={!selectedVariant?.is_available}
    style={{
      flex: 1, padding: '12px 24px',
      background: '#ffa500', color: '#131921',
      border: 'none', borderRadius: '4px',
      fontSize: '16px', fontWeight: 'bold',
      cursor: selectedVariant?.is_available ? 'pointer' : 'not-allowed',
    }}
  >
    [CART] В корзину
  </button>
</div>

{/* Описание */}
<div style={{ borderTop: '1px solid #eee', padding: '16px 0' }}>
  <h3>Описание</h3>
  <p style={{ lineHeight: '1.8' }}>{product.description}</p>
</div>

```

```

    { /* Теги */ }
    { product.tags && product.tags.length > 0 && (
      <div style={{ marginTop: '12px', display: 'flex', gap: '6px', flexWrap: 'wrap' }}>
        { product.tags.map(tag => (
          <span key={tag.slug} style={{
            background: '#f0f0f0', padding: '4px 10px',
            borderRadius: '12px', fontSize: '12px', color: '#555',
          }}>#{tag.name}</span>
        )) }
      </div>
    ) }
  </div>
</div>

{ /* == ОТЗЫВЫ == */ }
<div style={{ marginTop: '40px' }}>
  <h2>[CHAT] Отзывы ({product.reviews_count})</h2>

  { reviews.length === 0 ? (
    <p style={{ color: '#888', padding: '20px 0' }}>Пока нет отзывов</p>
  ) : (
    reviews.map(review => (
      <div key={review.id} style={{
        border: '1px solid #eee', borderRadius: '8px',
        padding: '16px', marginBottom: '12px',
      }}>
        <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '8px' }}>
          <div>
            <strong>{review.user.first_name || review.user.username}</strong>
            {review.verified_purchase && (
              <span style={{ color: '#007600', fontSize: '12px', marginLeft: '8px' }}>✓</span>
            ) }
          </div>
          <span style={{ color: '#888', fontSize: '12px' }}>
            {new Date(review.created_at).toLocaleDateString('ru-RU')}
          </span>
        </div>
        <div style={{ color: '#ffa500' }}>{'★'.repeat(review.rating)}{'☆'.repeat(5 - review.rating)}</div>
        <p style={{ marginTop: '8px', lineHeight: '1.6' }}>{review.text}</p>
        <div style={{ fontSize: '12px', color: '#888', marginTop: '8px' }}>
          [+]{review.helpful_yes} · [-]{review.helpful_no}
        </div>
      </div>
    ))
  ) }
</div>
);
}

```

Часть 9. Корзина

9.1 CartPage – полная страница корзины

[DOC] [DRF: AllowAny](#)

19 Создай файл `src/pages/CartPage.tsx`

```
// src/pages/CartPage.tsx
// Полная страница корзины: список товаров, изменение количества,
// удаление позиций, итогов, кнопка «Оформить заказ».
//
// ДАННЫЕ: все из cartStore (уже загружены в Layout)
// НЕ делаем дополнительный fetch – cartStore уже содержит корзину!

import { Link, useNavigate } from 'react-router-dom';
import { useCartStore } from '../stores/cartStore';

export function CartPage() {
  const { cart, isLoading, updateItem, removeItem, clearCart } = useCartStore();
  const navigate = useNavigate();

  if (isLoading) return <div style={{ textAlign: 'center', padding: '40px' }}>... Загрузка...</div>;

  if (!cart || cart.items.length === 0) {
    return (
      <div style={{ textAlign: 'center', padding: '60px' }}>
        <div style={{ fontSize: '60px', marginBottom: '16px' }}>[CART]</div>
        <h2>Корзина пуста</h2>
        <p style={{ color: '#888', marginBottom: '20px' }}>Добавьте товары из каталога</p>
        <Link to="/products" style={{
          padding: '12px 24px', background: '#ffa500', color: '#131921',
          textDecoration: 'none', borderRadius: '4px', fontWeight: 'bold',
        }}>Перейти в каталог</Link>
      </div>
    );
  }

  return (
    <div style={{ maxWidth: '900px', margin: '0 auto' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', margin-bottom: '20px' }}>
        <h1>[CART] Корзина ({cart.total_quantity})</h1>
        <button
          onClick={clearCart}
          style={{ padding: '8px 16px', background: 'none', border: '1px solid #ddd', borderRadius: '4px' }}>Очистить корзину</button>
      </div>

      {/* Список товаров */}
      <div style={{ marginBottom: '24px' }}>
        {cart.items.map(item => (
          <div key={item.id} style={{
            display: 'flex', gap: '16px', padding: '16px',
            border: '1px solid #eee', borderRadius: '8px', marginBottom: '12px',
            alignItems: 'center',
          }}>
            {/* Картинка */}
            <div style={{
              width: 80, height: 80, background: '#f5f5f5',
              borderRadius: '4px', display: 'flex', alignItems: 'center', justifyContent: 'center'
            }}></div>

```

```

        {item.image ? (
          <img src={item.image} alt={item.product_name} style={{ maxWidth: '100%', maxHeight: '100px' }} />
        ) : (
          <span style={{ fontSize: '32px' }}>[BOX]</span>
        )}
      </div>

      {/* Информация */}
      <div style={{ flex: 1 }}>
        <div style={{ fontWeight: 'bold' }}>{item.product_name}</div>
        <div style={{ color: '#888', fontSize: '12px' }}>{item.sku}</div>
        <div style={{ color: '#888', fontSize: '14px', marginTop: '4px' }}>{item.unit_price}</div>
      </div>

      {/* Количество */}
      <div style={{ display: 'flex', alignItems: 'center', gap: '8px' }}>
        <button onClick={() => updateItem(item.id, Math.max(1, item.quantity - 1))}
          style={{ width: 32, height: 32, border: '1px solid #ddd', borderRadius: '4px', background: 'white' }}>
          <span style={{ width: 30, textAlign: 'center' }}>{item.quantity}</span>
        <button onClick={() => updateItem(item.id, item.quantity + 1)}
          style={{ width: 32, height: 32, border: '1px solid #ddd', borderRadius: '4px', background: 'white' }}>
      </div>

      {/* Цена */}
      <div style={{ textAlign: 'right', minWidth: '100px' }}>
        <div style={{ fontWeight: 'bold', color: '#c00', fontSize: '18px' }}>{item.total}</div>
      </div>

      {/* Удалить */}
      <button
        onClick={() => removeItem(item.id)}
        style={{ background: 'none', border: 'none', color: 'red', cursor: 'pointer', font-size: '18px' }}>
        <span>✕</span>
      </button>
    </div>
  )}
</div>

{/* Итого */}
<div style={{
  background: '#f9f9f9', borderRadius: '8px', padding: '24px',
  border: '1px solid #eee',
}}>
  <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '12px' }}>
    <span>Товаров:</span>
    <span>{cart.total_quantity} шт.</span>
  </div>
  <div style={{ display: 'flex', justifyContent: 'space-between', fontSize: '24px', font-weight: 'bold' }}>
    <span>Итого:</span>
    <span style={{ color: '#c00' }}>{cart.total_price} ₺</span>
  </div>
  <button
    onClick={() => navigate('/checkout')}
    style={{

```



```
        width: '100%', marginTop: '20px', padding: '14px',
        background: '#ffa500', color: '#131921', border: 'none',
        borderRadius: '4px', fontSize: '18px', fontWeight: 'bold', cursor: 'pointer',
      }}
    >
      Оформить заказ →
    </button>
  </div>
</div>
);
}
```

Часть 10. Оформление заказа

10.1 OrderCreatePage – выбор адреса, доставка, купон

[DOC] [django-fsm](#)

20 Создай файл `src/pages/OrderCreatePage.tsx`

```
// src/pages/OrderCreatePage.tsx
// Страница оформления заказа: выбор адреса, способ доставки,
// применение купона, подтверждение.
//
// ЗАПРОСЫ К БЭКЕНДУ:
// 1. GET /users/addresses/           – список адресов
// 2. GET /shipping/methods/       – способы доставки
// 3. POST /shipping/calculate/      – расчёт стоимости доставки
// 4. POST /discounts/preview/       – предпросмотр скидки
// 5. POST /orders/                 – создание заказа

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import api from '../api/client';
import { USER, SHIPPING, DISCOUNT, ORDER, CART } from '../api/endpoints';
import { useCartStore } from '../stores/cartStore';

interface Address {
  id: number;
  recipient_name: string;
  city: string;
  street: string;
  postal_code: string;
  is_default: boolean;
}

interface ShippingMethod {
  id: number;
  name: string;
  description: string;
  estimated_days: string;
  price: string;
}

export function OrderCreatePage() {
  const { cart, fetchCart } = useCartStore();
  const navigate = useNavigate();

  const [addresses, setAddresses] = useState<Address[]>([]);
  const [shippingMethods, setShippingMethods] = useState<ShippingMethod[]>([]);
  const [selectedAddress, setSelectedAddress] = useState<number | null>(null);
  const [selectedShipping, setSelectedShipping] = useState<number | null>(null);
  const [couponCode, setCouponCode] = useState('');
  const [discount, setDiscount] = useState(0);
  const [deliveryCost, setDeliveryCost] = useState(0);
  const [comment, setComment] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [error, setError] = useState('');

  // Загружаем адреса и способы доставки
  useEffect(() => {
    api.get(USER.ADDRESSES).then(({ data }) => {
      setAddresses(data.results || data);
    });
  });
}
```

```

    const defaultAddr = (data.results || data).find((a: Address) => a.is_default);
    if (defaultAddr) setSelectedAddress(defaultAddr.id);
  });

  api.get(SHIPPING.METHODS).then(({ data }) => {
    setShippingMethods(data.results || data);
    if ((data.results || data).length > 0) setSelectedShipping(data[0]?.id || data.results[0].id);
  });
}, []);

// Рассчитываем стоимость доставки при изменении адреса/метода
useEffect(() => {
  if (selectedAddress && selectedShipping) {
    api.post(SHIPPING.CALCULATE, {
      address_id: selectedAddress,
      shipping_method_id: selectedShipping,
    }).then(({ data }) => {
      setDeliveryCost(parseFloat(data.cost || data.price || '0'));
    }).catch(() => setDeliveryCost(0));
  }
}, [selectedAddress, selectedShipping]);

// Применить купон
const handleApplyCoupon = async () => {
  if (!couponCode || !cart) return;
  try {
    const { data } = await api.post(DISCOUNT.PREVIEW, {
      order_id: cart.id, // временно используем cart.id
      code: couponCode,
    });
    setDiscount(parseFloat(data.discount_amount || data.discount || '0'));
  } catch {
    alert('Купон недействителен');
    setDiscount(0);
  }
};

// Оформить заказ
const handleSubmit = async () => {
  if (!selectedAddress || !selectedShipping) {
    setError('Выберите адрес и способ доставки');
    return;
  }

  setIsSubmitting(true);
  setError('');

  try {
    const { data: order } = await api.post(ORDER.LIST, {
      address_id: selectedAddress,
      shipping_method_id: selectedShipping,
      comment,
    });
  }

```

```

// Применяем купон, если есть
if (couponCode && discount > 0) {
  await api.post(DISCOUNT.APPLY, {
    order_id: order.id,
    code: couponCode,
  });
}

// Обновляем корзину (она должна стать пустой)
await fetchCart();

// Переходим на страницу заказа
navigate(`/orders/${order.order_number}`, { replace: true });
} catch (err: any) {
  setError(err.response?.data?.detail || 'Ошибка при оформлении заказа');
} finally {
  setIsSubmitting(false);
}
};

if (!cart || cart.items.length === 0) {
  return <div style={{ textAlign: 'center', padding: '40px' }}>Корзина пуста. <a href="/products" >Перейти к товарам</a></div>
}

const subtotal = parseFloat(cart.total_price);
const total = subtotal + deliveryCost - discount;

return (
  <div style={{ maxWidth: '900px', margin: '0 auto' }}>
    <h1 style={{ marginBottom: '24px' }}>[LIST] Оформление заказа</h1>

    {error && <div style={{ background: '#fce4ec', color: '#c62828', padding: '12px', border: '1px solid #c62828' }}>{error}</div>}

    <div style={{ display: 'grid', gridTemplateColumns: '1fr 350px', gap: '24px' }}>
      <div>
        {/* Адрес доставки */}
        <div style={{ border: '1px solid #eee', borderRadius: '8px', padding: '20px', margin: '0 auto' }}>
          <h2>[PIN] Адрес доставки</h2>
          {addresses.length === 0 ? (
            <p style={{ color: '#888' }}>Нет сохранённых адресов. Добавьте в <a href="/profile" >Профиль</a></p>
          ) : (
            addresses.map(addr => (
              <label key={addr.id} style={{ display: 'block', padding: '8px', border: '1px solid #eee' }}>
                <input type="radio" name="address" checked={selectedAddress === addr.id} onChange={setSelectedAddress(addr.id)} />
                <strong>{addr.recipient_name}</strong>
                {addr.is_default && <span style={{ color: '#007600', fontSize: '12px', margin-left: '10px' }}>Адрес по умолчанию</span>}
                <div style={{ color: '#666', fontSize: '14px' }}>{addr.city}, {addr.street}, {addr.postcode}</div>
              </label>
            ))
          )}
        </div>
      </div>
    </div>
  </div>
);

```

```

    { /* Способ доставки */ }
    <div style={{ border: '1px solid #eee', borderRadius: '8px', padding: '20px', margin: '10px 0' }}>
      <h2>[TRUCK] Способ доставки</h2>
      {shippingMethods.map(method => (
        <label key={method.id} style={{ display: 'block', padding: '8px', border: '1px solid #ccc', margin: '4px 0' }}>
          <input type="radio" name="shipping" checked={selectedShipping === method.id} />
          <strong>{method.name}</strong> – {method.price} ₽
          <div style={{ color: '#666', fontSize: '14px' }}>{method.description} · {method.deliveryTime}
        </label>
      ))}
    </div>

    { /* Комментарий */ }
    <div style={{ border: '1px solid #eee', borderRadius: '8px', padding: '20px' }}>
      <h2>[CHAT] Комментарий</h2>
      <textarea
        value={comment}
        onChange={e => setComment(e.target.value)}
        placeholder="Позвонить перед доставкой..."
        style={{ width: '100%', padding: '10px', border: '1px solid #ccc', borderRadius: '4px' }} />
    </div>
  </div>

  { /* Правая колонка – итог */ }
  <div>
    <div style={{ border: '1px solid #eee', borderRadius: '8px', padding: '20px', position: 'relative' }}>
      <h2>[RECEIPT] Ваш заказ</h2>

      {cart.items.map(item => (
        <div key={item.id} style={{ display: 'flex', justifyContent: 'space-between', font-family: 'monospace' }}>
          <span>{item.product_name} × {item.quantity}</span>
          <span>{item.total_price} ₽</span>
        </div>
      ))}

      <hr style={{ border: 'none', borderTop: '1px solid #eee', margin: '12px 0' }} />

      <div style={{ display: 'flex', justifyContent: 'space-between', margin: '10px 0' }}>
        <span>Подытог:</span>
        <span>{subtotal.toFixed(2)} ₽</span>
      </div>

      <div style={{ display: 'flex', justifyContent: 'space-between', margin: '10px 0' }}>
        <span>Доставка:</span>
        <span>{deliveryCost.toFixed(2)} ₽</span>
      </div>

      {discount > 0 && (
        <div style={{ display: 'flex', justifyContent: 'space-between', margin: '10px 0' }}>
          <span>Скидка:</span>
          <span>–{discount.toFixed(2)} ₽</span>
        </div>
      )}

      <hr style={{ border: 'none', borderTop: '2px solid #131921', margin: '12px 0' }} />
      <div style={{ display: 'flex', justifyContent: 'space-between', font-size: '20px', font-weight: 'bold' }}>
        <span>Итого:</span>
        <span style={{ color: '#c00' }}>{total.toFixed(2)} ₽</span>
      </div>
    </div>
  </div>

```

```

    { /* Купон */ }
    <div style={{ marginTop: '16px' }}>
      <div style={{ display: 'flex', gap: '8px' }}>
        <input
          value={couponCode}
          onChange={e => setCouponCode(e.target.value)}
          placeholder="Промокод"
          style={{ flex: 1, padding: '8px', border: '1px solid #ccc', borderRadius: '4px' }}
        />
        <button onClick={handleApplyCoupon} style={{ padding: '8px 12px', background: 'white', border: '1px solid #ccc', borderRadius: '4px' }}>
          Применить
        </button>
      </div>
    </div>

    <button
      onClick={handleSubmit}
      disabled={isSubmitting}
      style={{
        width: '100%', marginTop: '16px', padding: '14px',
        background: '#ffa500', color: '#131921', border: 'none',
        borderRadius: '4px', fontSize: '16px', fontWeight: 'bold',
        cursor: isSubmitting ? 'not-allowed' : 'pointer',
      }}
    >
      {isSubmitting ? 'Оформляем...' : 'Подтвердить заказ'}
    </button>
  </div>
</div>
</div>
</div>
);
}

```

10.2 OrderDetailPage – детали заказа

21 Создай файл `src/pages/OrderDetailPage.tsx`

```
// src/pages/OrderDetailPage.tsx
// Детали одного заказа: состав, статус, стоимость, отслеживание.

import { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';
import api from '../api/client';
import { ORDER } from '../api/endpoints';

interface OrderItem {
  variant_id: number;
  product_name: string;
  sku: string;
  unit_price: string;
  quantity: number;
  total_price: string;
}

interface Order {
  order_number: string;
  status: string;
  items: OrderItem[];
  subtotal: string;
  delivery_cost: string;
  discount: string;
  total: string;
  comment: string;
  created_at: string;
}

// Цвета для статусов
const STATUS_COLORS: Record<string, string> = {
  PENDING: '#ff9800',
  CONFIRMED: '#2196f3',
  PROCESSING: '#9c27b0',
  SHIPPED: '#4caf50',
  DELIVERED: '#007600',
  CANCELLED: '#f44336',
};

const STATUS_LABELS: Record<string, string> = {
  PENDING: '... Ожидает оплаты',
  CONFIRMED: 'ДА Подтверждён',
  PROCESSING: 'В обработке',
  SHIPPED: '[TRUCK] Отправлен',
  DELIVERED: '[BOX] Доставлен',
  CANCELLED: 'HET Отменён',
};

export function OrderDetailPage() {
  const { orderNumber } = useParams<{ orderNumber: string }>();
  const [order, setOrder] = useState<Order | null>(null);
  const [isLoading, setIsLoading] = useState(true);

```



```

useEffect(() => {
  if (!orderNumber) return;
  api.get(ORDER.DETAIL(orderNumber))
    .then(({ data }) => setOrder(data))
    .catch(() => setOrder(null))
    .finally(() => setIsLoading(false));
}, [orderNumber]);

const handleCancel = async () => {
  if (!orderNumber || !confirm('Отменить заказ?')) return;
  try {
    await api.post(ORDER.CANCEL(orderNumber));
    setOrder(prev => prev ? { ...prev, status: 'CANCELLED' } : null);
  } catch {
    alert('Не удалось отменить заказ');
  }
};

if (isLoading) return <div style={{ textAlign: 'center', padding: '40px' }}>... Загрузка...</div>
if (!order) return <div style={{ textAlign: 'center', padding: '40px' }}>Заказ не найден</div>

return (
  <div style={{ maxWidth: '700px', margin: '0 auto' }}>
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', margin-bottom: '20px' }}>
      <h1>[BOX] Заказ {order.order_number}</h1>
      {(order.status === 'PENDING' || order.status === 'CONFIRMED') && (
        <button onClick={handleCancel} style={{ padding: '8px 16px', background: '#f44336', color: 'white' }}>Отменить</button>
      )}
    </div>

    { /* Статус */ }
    <div style={{
      padding: '16px', borderRadius: '8px', marginBottom: '20px',
      background: STATUS_COLORS[order.status] + '20',
      border: `2px solid ${STATUS_COLORS[order.status]}`,
    }}>
      <div style={{ fontSize: '18px', fontWeight: 'bold', color: STATUS_COLORS[order.status] }}>
        {STATUS_LABELS[order.status] || order.status}
      </div>
      <div style={{ color: '#666', fontSize: '14px', marginTop: '4px' }}>
        от {new Date(order.created_at).toLocaleString('ru-RU')}
      </div>
    </div>

    { /* Состав заказа */ }
    <div style={{ border: '1px solid #eee', borderRadius: '8px', overflow: 'hidden', marginBottom: '20px' }}>
      <div style={{ background: '#f5f5f5', padding: '12px 16px', fontWeight: 'bold' }}>Состав заказа</div>
      {order.items.map((item, i) => (
        <div key={i} style={{ display: 'flex', justifyContent: 'space-between', padding: '12px 16px' }}>
          <div>
            <div style={{ fontWeight: 'bold' }}>{item.product_name}</div>
            <div style={{ color: '#888', fontSize: '14px' }}>{item.sku} · {item.unit_price} $
          </div>
        </div>
      ))}
    </div>
  </div>
)

```

```

        <div style={{ fontWeight: 'bold', color: '#c00' }}>{item.total_price} ₽</div>
      </div>
    )}}
  </div>

  {/* Итого */}
  <div style={{ border: '1px solid #eee', borderRadius: '8px', padding: '20px' }}>
    <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '8px' }}>
      <span>Подытог:</span><span>{order.subtotal} ₽</span>
    </div>
    <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '8px' }}>
      <span>Доставка:</span><span>{order.delivery_cost} ₽</span>
    </div>
    {parseFloat(order.discount) > 0 && (
      <div style={{ display: 'flex', justifyContent: 'space-between', marginBottom: '8px',
        <span>Скидка:</span><span>--{order.discount} ₽</span>
      </div>
    )}
    <hr style={{ border: 'none', borderTop: '2px solid #131921', margin: '12px 0' }} />
    <div style={{ display: 'flex', justifyContent: 'space-between', fontSize: '20px', fontW
      <span>Итого:</span><span style={{ color: '#c00' }}>{order.total} ₽</span>
    </div>
  </div>

  {order.comment && (
    <div style={{ marginTop: '16px', color: '#666' }}>
      <strong>Комментарий:</strong> {order.comment}
    </div>
  )}

  <div style={{ marginTop: '24px' }}>
    <Link to="/orders" style={{ color: '#ffa500' }}>← Все заказы</Link>
  </div>
</div>
);
}

```

10.3 OrderListPage – список заказов

22 Создай файл `src/pages/OrderListPage.tsx`

```
// src/pages/OrderListPage.tsx
// Список всех заказов пользователя.

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import api from '../api/client';
import { ORDER } from '../api/endpoints';

interface OrderListItem {
  order_number: string;
  status: string;
  total: string;
  items_count: number;
  created_at: string;
}

export function OrderListPage() {
  const [orders, setOrders] = useState<OrderListItem[]>([]);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    api.get(ORDER.LIST)
      .then(({ data }) => setOrders(data.results || data))
      .finally(() => setIsLoading(false));
  }, []);

  const STATUS_LABELS: Record<string, string> = {
    PENDING: '... Ожидает', CONFIRMED: 'ДА Подтверждён',
    PROCESSING: 'В обработке', SHIPPED: '[TRUCK] Отправлен',
    DELIVERED: '[BOX] Доставлен', CANCELLED: 'НЕТ Отменён',
  };

  return (
    <div style={{ maxWidth: '700px', margin: '0 auto' }}>
      <h1 style={{ marginBottom: '24px' }}>[BOX] Мои заказы</h1>

      {isLoading ? <div>... Загрузка...</div> : orders.length === 0 ? (
        <div style={{ textAlign: 'center', padding: '40px' }}>
          <div style={{ fontSize: '48px', marginBottom: '12px' }}>[EMPTY]</div>
          <p style={{ color: '#888' }}>У вас пока нет заказов</p>
          <Link to="/products" style={{ color: '#ffa500' }}>Перейти в каталог</Link>
        </div>
      ) : (
        orders.map(order => (
          <Link key={order.order_number} to={`/${orders}/${order.order_number}`}
            style={{ display: 'block', textDecoration: 'none', color: 'inherit',
              border: '1px solid #eee', borderRadius: '8px', padding: '16px',
              marginBottom: '12px', transition: 'box-shadow 0.2s' }}>
            <div style={{ display: 'flex', justifyContent: 'space-between' }}>
              <div>
                <strong>{order.order_number}</strong>
                <span style={{ marginLeft: '12px', fontSize: '14px' }}>
                  {STATUS_LABELS[order.status] || order.status}
                
```

```
        </span>
      </div>
      <div style={{ fontWeight: 'bold', color: '#c00' }}>{order.total} ₽</div>
    </div>
    <div style={{ color: '#888', fontSize: '14px', marginTop: '4px' }}>
      {new Date(order.created_at).toLocaleString('ru-RU')} · {order.items_count} товаров
    </div>
  </Link>
))
})
</div>
);
}
```

Часть 11. Личный кабинет

11.1 ProfilePage – редактирование профиля

23 Создай файл `src/pages/ProfilePage.tsx`

```
// src/pages/ProfilePage.tsx
// Личный кабинет: редактирование профиля + управление адресами.
//
// ЗАПРОСЫ К БЭКЕНДУ:
//   GET/PATCH /users/me/           – профиль
//   GET/POST /users/addresses/     – адреса
//   DELETE /users/addresses/{id}/ – удалить адрес
//   POST /users/addresses/{id}/default/ – сделать основным

import { useEffect, useState } from 'react';
import api from '../api/client';
import { USER } from '../api/endpoints';
import { useAuthStore } from '../stores/authStore';

interface Address {
  id: number;
  recipient_name: string;
  city: string;
  street: string;
  postal_code: string;
  is_default: boolean;
}

export function ProfilePage() {
  const { user, fetchUser } = useAuthStore();
  const [addresses, setAddresses] = useState<Address[]>([]);
  const [isEditing, setIsEditing] = useState(false);
  const [editData, setEditData] = useState({
    first_name: user?.first_name || '',
    last_name: user?.last_name || '',
    phone: user?.phone || '',
  });
  const [newAddress, setNewAddress] = useState({
    recipient_name: '', city: '', street: '', postal_code: '',
  });
  const [showAddressForm, setShowAddressForm] = useState(false);

  useEffect(() => {
    api.get(USER.ADDRESSES).then(({ data }) => setAddresses(data.results || data));
  }, []);

  const handleSaveProfile = async () => {
    try {
      await api.patch(USER.ME, editData);
      await fetchUser(); // Обновляем глобальное состояние
      setIsEditing(false);
    } catch {
      alert('Ошибка при сохранении');
    }
  };

  const handleAddAddress = async (e: React.FormEvent) => {
    e.preventDefault();
  }
}
```

```

    try {
      await api.post(USER.ADDRESSES, newAddress);
      const { data } = await api.get(USER.ADDRESSES);
      setAddresses(data.results || data);
      setNewAddress({ recipient_name: '', city: '', street: '', postal_code: '' });
      setShowAddressForm(false);
    } catch {
      alert('Ошибка при добавлении адреса');
    }
  };

const handleDeleteAddress = async (id: number) => {
  if (!confirm('Удалить адрес?')) return;
  try {
    await api.delete(USER.ADDRESS(id));
    setAddresses(prev => prev.filter(a => a.id !== id));
  } catch {
    alert('Ошибка при удалении');
  }
};

const handleSetDefault = async (id: number) => {
  try {
    await api.post(USER.ADDRESS_DEFAULT(id));
    setAddresses(prev => prev.map(a => ({ ...a, is_default: a.id === id })));
  } catch {
    alert('Ошибка');
  }
};

if (!user) return <div>... Загрузка...</div>;

return (
  <div style={{ maxWidth: '700px', margin: '0 auto' }}>
    <h1 style={{ marginBottom: '24px' }}>[USER] Личный кабинет</h1>

    {/* Профиль */}
    <div style={{ border: '1px solid #eee', borderRadius: '8px', padding: '20px', marginBottom: '20px' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', margin-bottom: '10px' }}>
        <h2>[LIST] Профиль</h2>
        <button onClick={() => setIsEditing(!isEditing)}
          style={{ padding: '6px 12px', background: 'none', border: '1px solid #ffa500', border-radius: '4px' }}>
          {isEditing ? 'Отмена' : 'Редактировать'}
        </button>
      </div>
    </div>

    {isEditing ? (
      <div style={{ display: 'grid', gap: '12px' }}>
        <div>
          <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Имя
          <input value={editData.first_name} onChange={e => setEditData({...editData, first_name: e.target.value})}
            style={{ width: '100%', padding: '8px', border: '1px solid #ccc', borderRadius: '4px' }} />
        </div>
      </div>
    ) : null}
  </div>
);

```

```

    <div>
      <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Фамилия</label>
      <input value={editData.last_name} onChange={e => setEditData({...editData, last_name: e.target.value})}
        style={{ width: '100%', padding: '8px', border: '1px solid #ccc', borderRadius: '4px' }} />
    </div>
    <div>
      <label style={{ display: 'block', marginBottom: '4px', fontWeight: 'bold' }}>Телефон</label>
      <input value={editData.phone} onChange={e => setEditData({...editData, phone: e.target.value})}
        style={{ width: '100%', padding: '8px', border: '1px solid #ccc', borderRadius: '4px' }} />
    </div>
    <button onClick={handleSaveProfile}
      style={{ padding: '10px', background: '#ffa500', border: 'none', borderRadius: '4px' }}>Сохранить</button>
  </div>
) : (
  <div>
    <div style={{ fontSize: '20px', fontWeight: 'bold', marginBottom: '8px' }}>{user.first_name}</div>
    <div style={{ color: '#666' }}>[MAIL] {user.email}</div>
    <div style={{ color: '#666' }}>[USER] @{user.username}</div>
    {user.phone && <div style={{ color: '#666' }}>[PHONE] {user.phone}</div>}
  </div>
)
</div>

{/* Адреса */}
<div style={{ border: '1px solid #eee', borderRadius: '8px', padding: '20px' }}>
  <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', margin-bottom: '10px' }}>
    <h2>[PIN] Адреса доставки</h2>
    <button onClick={() => setShowAddressForm(!showAddressForm)}
      style={{ padding: '6px 12px', background: '#ffa500', border: 'none', borderRadius: '4px' }}>Добавить</button>
  </div>

  {showAddressForm && (
    <form onSubmit={handleAddAddress} style={{ background: '#f9f9f9', padding: '16px', border: '1px solid #eee' }}>
      <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: '12px' }}>
        <input placeholder="ФИО получателя" value={newAddress.recipient_name} onChange={e => setNewAddress({ ...newAddress, recipient_name: e.target.value })} />
        <input placeholder="Город" value={newAddress.city} onChange={e => setNewAddress({ ...newAddress, city: e.target.value })} />
        <input placeholder="Улица, дом, кв." value={newAddress.street} onChange={e => setNewAddress({ ...newAddress, street: e.target.value })} />
        <input placeholder="Почтовый индекс" value={newAddress.postal_code} onChange={e => setNewAddress({ ...newAddress, postal_code: e.target.value })} />
      </div>
      <button type="submit" style={{ marginTop: '12px', padding: '8px 16px', background: '#ffa500', border: 'none', borderRadius: '4px' }}>Сохранить</button>
    </form>
  )}

  {addresses.length === 0 ? (
    <p style={{ color: '#888' }}>Нет сохранённых адресов</p>
  ) : (
    addresses.map(addr => (
      <div key={addr.id} style={{ display: 'flex', justifyContent: 'space-between', align-items: 'center', margin-bottom: '10px' }}>
        <div>
          <strong>{addr.recipient_name}</strong>
          {addr.is_default && <span style={{ color: '#007600', marginLeft: '8px', fontSize: '14px' }}>по умолчанию</span>}
          <div style={{ color: '#666', fontSize: '14px' }}>{addr.city}, {addr.street}, {addr.postal_code}</div>
        </div>
        <button onClick={() => deleteAddress(addr.id)} style={{ color: '#f44336', border: '1px solid #f44336', padding: '2px 8px', borderRadius: '4px' }}>Удалить</button>
      </div>
    ))
  )}
</div>

```



```
        <div style={{ display: 'flex', gap: '8px' }}>
          {!addr.is_default && (
            <button onClick={() => handleSetDefault(addr.id)} style={{ padding: '4px 8px' }}>
              Set Default
            </button>
            <button onClick={() => handleDeleteAddress(addr.id)} style={{ background: 'none' }}>
              Delete
            </button>
          </div>
        </div>
      ))
    )}
  </div>
</div>
);
}
```

Часть 12. Избранное и уведомления

12.1 WishlistPage – список желаний

24 Создай файл `src/pages/WishlistPage.tsx`

```

// src/pages/WishlistPage.tsx
// Страница списка желаний (избранное).
// Требуется авторизации.

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { useWishlistStore } from '../stores/wishlistStore';
import { useCartStore } from '../stores/cartStore';

export function WishlistPage() {
  const { wishlist, fetchWishlist, removeItem, moveToCart } = useWishlistStore();
  const { addItem } = useCartStore();
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    fetchWishlist().finally(() => setIsLoading(false));
  }, []);

  const handleMoveToCart = async (itemId: number, variantId: number) => {
    try {
      await moveToCart(itemId);
    } catch {
      // Если moveToCart не добавляет в корзину автоматически:
      await addItem(variantId);
      await removeItem(itemId);
    }
  };

  if (isLoading) return <div style={{ textAlign: 'center', padding: '40px' }}>... Загрузка...</div>

  return (
    <div style={{ maxWidth: '700px', margin: '0 auto' }}>
      <h1 style={{ marginBottom: '24px' }}>♥ Избранное ({wishlist?.items_count || 0})</h1>

      {!wishlist || wishlist.items.length === 0 ? (
        <div style={{ textAlign: 'center', padding: '40px' }}>
          <div style={{ fontSize: '48px', marginBottom: '12px' }}>[HEART]</div>
          <p style={{ color: '#888' }}>Список желаний пуст</p>
          <Link to="/products" style={{ color: '#ffa500' }}>Перейти в каталог</Link>
        </div>
      ) : (
        wishlist.items.map(item => (
          <div key={item.id} style={{
            display: 'flex', justifyContent: 'space-between', alignItems: 'center',
            padding: '16px', border: '1px solid #eee', borderRadius: '8px', marginBottom: '12px'
          }}>
            <div>
              <div style={{ fontWeight: 'bold' }}>{item.product_name}</div>
              <div style={{ color: '#888', fontSize: '14px' }}>Добавлено {new Date(item.added_at).toLocaleDateString()}</div>
            </div>
            <div style={{ display: 'flex', gap: '8px' }}>
              <button
                onClick={() => handleMoveToCart(item.id, item.variant_id)}

```

```

        style={{ padding: '6px 12px', background: '#ffa500', border: 'none', borderRad
    >[CART] В корзину</button>
    <button
      onClick={() => removeItem(item.id)}
      style={{ padding: '6px 12px', background: 'none', border: '1px solid #ddd', bo
    >x</button>
  </div>
</div>
))
)}
</div>
);
}

```

12.2 NotificationBell – колокольчик уведомлений

Колокольчик уже встроен в `Layout.tsx`. Вот отдельная страница уведомлений:

```

// src/pages/NotificationsPage.tsx (опционально)
// Страница со списком всех уведомлений.

import { useEffect, useState } from 'react';
import { useNotificationStore } from '../stores/notificationStore';

export function NotificationsPage() {
  const { notifications, fetchNotifications, markRead, markAllRead, unreadCount } = useNotificationStore();
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    fetchNotifications().finally(() => setIsLoading(false));
  }, []);

  if (isLoading) return <div style={{ textAlign: 'center', padding: '40px' }}>... Загрузка...</div>

  return (
    <div style={{ maxWidth: '600px', margin: '0 auto' }}>
      <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', margin-bottom: '10px' }}>
        <h1>[BELL] Уведомления {unreadCount > 0 && <span style={{ fontSize: '16px', color: '#f44336' }}>{unreadCount} </span>}</h1>
        <button onClick={markAllRead} style={{ padding: '6px 12px', background: '#37475a', color: 'white' }}>
          {unreadCount > 0 && 'Очистить'}
        </button>
      </div>

      {notifications.length === 0 ? (
        <div style={{ textAlign: 'center', padding: '40px', color: '#888' }}>Нет уведомлений</div>
      ) : (
        notifications.map(n => (
          <div key={n.id} style={{
            padding: '12px 16px', border: '1px solid #eee',
            borderRadius: '4px', marginBottom: '8px',
            background: n.is_read ? 'white' : '#fff8e1',
          }}>
            <div style={{ display: 'flex', justifyContent: 'space-between' }}>
              <strong>{n.title}</strong>
              <span style={{ fontSize: '12px', color: '#888' }}>{new Date(n.created_at).toLocaleDateString()}</span>
            </div>
            <div style={{ color: '#666', marginTop: '4px' }}>{n.message}</div>
            {!n.is_read && (
              <button onClick={() => markRead(n.id)} style={{ marginTop: '8px', padding: '4px 8px' }}>
                Прочитать
              </button>
            )}
          </div>
        ))
      )}
    </div>
  );
}

```

Часть 13. Главная страница

13.1 HomePage – hero-баннер + популярные товары

25 Создай файл `src/pages/HomePage.tsx`

```
// src/pages/HomePage.tsx
// Главная страница: hero-баннер + популярные товары + категории.

import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import api from '../api/client';
import { CATALOG } from '../api/endpoints';

interface Product {
  id: string;
  name: string;
  slug: string;
  min_price: string;
  rating: string;
  reviews_count: number;
  main_image: string | null;
  brand: { name: string } | null;
}

export function HomePage() {
  const [featured, setFeatured] = useState<Product[]>([]);

  useEffect(() => {
    api.get(CATALOG.PRODUCTS, { params: { page_size: 8, ordering: '-rating' } })
      .then(({ data }) => setFeatured(data.results || []))
      .catch(() => {});
  }, []);

  return (
    <div>
      {/* == Hero-баннер == */}
      <div style={{
        background: 'linear-gradient(135deg, #131921 0%, #232f3e 100%)',
        color: 'white', padding: '60px 40px', borderRadius: '8px',
        marginBottom: '32px', textAlign: 'center',
      }}>
        <h1 style={{ fontSize: '36px', marginBottom: '12px' }}>Добро пожаловать в Amazone</h1>
        <p style={{ fontSize: '18px', color: '#ffa500', marginBottom: '24px' }}>Лучшие товары п
        <Link to="/products">
          <button style={{
            padding: '14px 32px', background: '#ffa500', border: 'none',
            borderRadius: '4px', fontSize: '18px', cursor: 'pointer', fontWeight: 'bold',
          }}>Перейти в каталог →</button>
        </Link>
      </div>

      {/* == Популярные товары == */}
      <h2 style={{ marginBottom: '16px' }}>[FIRE] Популярные товары</h2>

      <div style={{
        display: 'grid',
        gridTemplateColumns: 'repeat(auto-fill, minmax(220px, 1fr))',
        gap: '16px',

```

```

}}>
{featured.map(product => (
  <Link key={product.id} to={`/${product.slug}`}
    style={{ textDecoration: 'none', color: 'inherit', border: '1px solid #ddd', border
  <div style={{
    height: '160px', background: '#f5f5f5', borderRadius: '4px',
    display: 'flex', alignItems: 'center', justifyContent: 'center', marginBottom: '8
  }}>
    {product.main_image ? (
      <img src={product.main_image} alt={product.name} style={{ maxWidth: '100%', max
    ) : (
      <span style={{ fontSize: '48px' }}>[BOX]</span>
    )}
  </div>
  <div style={{ fontWeight: 'bold', fontSize: '14px', marginBottom: '4px' }}>{product
  {product.brand && <div style={{ color: '#888', fontSize: '12px' }}>{product.brand.n
  <div style={{ color: '#c00', fontWeight: 'bold', marginTop: '4px' }}>{product.min_p
  <div style={{ fontSize: '12px', color: '#666' }}>★ {parseFloat(product.rating).toFi
  </Link>
  )})}
</div>
</div>
);
}

```

Глобальные стили – обнови src/index.css


```
/* src/index.css */
/* Базовые стили для всего приложения */

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  color: #0f1111;
  background: #eaeaed;
}

a {
  text-decoration: none;
}

button:hover {
  opacity: 0.9;
}

input:focus, textarea:focus, select:focus {
  outline: 2px solid #ffa500;
  border-color: #ffa500 !important;
}

/* Smooth transitions */
button, a, input {
  transition: all 0.15s ease;
}
```

Часть 14. TypeScript-типы для API

TypeScript – это «JavaScript с проверкой типов». Как `myru` в Python. Он ловит ошибки до запуска кода.

[!] Зачем типы?

Когда ты пишешь `product.nme` вместо `product.name`, TypeScript подчёркнёт ошибку красным СРАЗУ. Без типов ты бы увидел «undefined» только в браузере, потратив 30 минут на отладку.

26 Создай файл `src/types/index.ts`

```
// src/types/index.ts
// Все TypeScript-интерфейсы для API-ответов Django.
// Один файл — чтобы не искать тип по всему проекту.

// =====
// Auth
// =====

/** Ответ POST /auth/login/ */
export interface TokenPair {
  access: string;
  refresh: string;
}

/** Ответ POST /auth/register/ */
export interface RegisterResponse {
  id: number;
  email: string;
  username: string;
  first_name: string;
  last_name: string;
}

// =====
// Users
// =====

export interface UserProfile {
  timezone: string;
  language: string;
  avatar: string | null;
}

/** Ответ GET /users/me/ */
export interface User {
  id: number;
  email: string;
  username: string;
  first_name: string;
  last_name: string;
  phone: string;
  profile: UserProfile | null;
}

export interface Address {
  id: number;
  recipient_name: string;
  city: string;
  street: string;
  postal_code: string;
  is_default: boolean;
  created_at: string;
}
```

```

// =====
// Catalog
// =====

export interface Brand {
  name: string;
  slug: string;
}

export interface Category {
  name: string;
  slug: string;
  children?: Category[];
}

export interface Variant {
  id: number;
  name: string;
  sku: string;
  price: string;
  sale_price: string | null;
  effective_price: string;
  is_available: boolean;
  attributes: Record<string, string>;
}

export interface ProductImage {
  id: number;
  image: string;
  alt: string;
  is_main: boolean;
}

export interface Tag {
  name: string;
  slug: string;
}

/** Товар в списке (GET /catalog/products/) */
export interface ProductListItem {
  id: string;
  name: string;
  slug: string;
  min_price: string;
  max_price: string;
  rating: string;
  reviews_count: number;
  views_count: number;
  main_image: string | null;
  brand: Brand | null;
  category: Category | null;
}

```

```

/** Детали товара (GET /catalog/products/{slug}/) */
export interface ProductDetail extends ProductListItem {
  description: string;
  images: ProductImage[];
  variants: Variant[];
  tags: Tag[];
}

// =====
// Cart
// =====

export interface CartItem {
  id: number;
  variant_id: number;
  product_name: string;
  sku: string;
  unit_price: string;
  quantity: number;
  total_price: string;
  image: string | null;
}

export interface Cart {
  id: number;
  items: CartItem[];
  total_quantity: number;
  total_price: string;
}

// =====
// Orders
// =====

export type OrderStatus =
  | 'PENDING' | 'CONFIRMED' | 'PROCESSING'
  | 'SHIPPED' | 'DELIVERED' | 'CANCELLED';

export interface OrderItem {
  variant_id: number;
  product_name: string;
  sku: string;
  unit_price: string;
  quantity: number;
  total_price: string;
}

export interface Order {
  order_number: string;
  status: OrderStatus;
  items: OrderItem[];
  subtotal: string;
}

```

```

    delivery_cost: string;
    discount: string;
    total: string;
    comment: string;
    created_at: string;
  }

// =====
// Reviews
// =====

export interface Review {
  id: number;
  user: { username: string; first_name: string };
  rating: number;
  text: string;
  verified_purchase: boolean;
  helpful_yes: number;
  helpful_no: number;
  created_at: string;
}

// =====
// Wishlist
// =====

export interface WishlistItem {
  id: number;
  variant_id: number;
  product_name: string;
  added_at: string;
}

export interface Wishlist {
  id: number;
  items: WishlistItem[];
  items_count: number;
}

// =====
// Notifications
// =====

export interface Notification {
  id: number;
  type: string;
  title: string;
  message: string;
  is_read: boolean;
  created_at: string;
}

// =====

```

```
// Shipping
// =====

export interface ShippingMethod {
  id: number;
  name: string;
  description: string;
  estimated_days: string;
  price: string;
}

// =====
// Paginated response (стандартный формат Django)
// =====

export interface PaginatedResponse<T> {
  count: number;
  next: string | null;
  previous: string | null;
  results: T[];
}
```

Часть 15. Тестирование и отладка

15.1 Проверка каждого эндпоинта вручную

Перед тем как запускать React, убедись, что бэкенд отвечает правильно:

Шаг 1: Запусти Django

```
cd I:\NewPythonProjects\Amazon_Clonе  
python manage.py runserver
```

Шаг 2: Проверь health-check

```
# Открой в браузере:  
http://localhost:8000/api/v1/health/  
# Должно быть: {"status":"ok","version":"1.0.0","database":"ok"}
```

Шаг 3: Проверь Swagger

```
# Открой в браузере:  
http://localhost:8000/api/v1/docs/  
# Должна открыться интерактивная документация
```

Шаг 4: Зарегистрируй пользователя через Swagger

```
# Или через curl / PowerShell:  
# PowerShell:  
Invoke-RestMethod -Method Post -Uri "http://localhost:8000/api/v1/auth/register/" -ContentType
```

Шаг 5: Получи JWT-токен


```
# PowerShell:
$response = Invoke-RestMethod -Method Post -Uri "http://localhost:8000/api/v1/auth/login/" -Cor
$response.access # → eyJhbG...
$response.refresh # → eyJhbG...
```

Шаг 6: Запусти React

```
cd I:\NewPythonProjects\frontend
npm run dev
```

Шаг 7: Проверь в React

1. Открой <http://localhost:5173> → Главная страница
2. Нажми «Войти» → Страница логина
3. Введи email + пароль → Должен войти
4. Хедер должен показать имя пользователя
5. Нажми «Каталог» → Список товаров
6. Нажми на товар → Карточка товара

15.2 Типичные ошибки и их решения

[\[DOC\] React DevTools](#)

Ошибка	Причина	Решение
Network Error	Django не запущен	Запусти <code>python manage.py runserver</code>
CORS error	CORS не настроен	Проверь <code>CorsMiddleware</code> в <code>settings.py</code>
401 Unauthorized на каждый запрос	access-токен не подставляется	Проверь request-интерцептор в <code>client.ts</code>
401 закидывается	refresh-токен тоже истёк	response-интерцептор должен чистить <code>localStorage</code> при ошибке <code>refresh</code>
Страницы не переключаются	Не подключён <code>RouterProvider</code>	Проверь <code>main.tsx</code> : <code><RouterProvider router={router} /></code>
Защищённые страницы редиректят на <code>/login</code>	<code>fetchUser</code> ещё не завершился	Проверь <code>isLoading</code> в <code>ProtectedRoute</code>
Корзина пуста после обновления	Гостевая корзина потерялась	Нужен <code>cart/merge/</code> после логина
ECONNREFUSED	Vite проху не может подключиться к Django	Проверь порт 8000 и <code>vite.config.ts</code>
Картинки не грузятся	Media-прокси не настроен	Добавь <code>/media</code> в проху в <code>vite.config.ts</code>
<code>TypeError: Cannot read properties of undefined</code>	Ответ API не соответствует ожидаемой структуре	Проверь ответ в Swagger. Добавь <code>?.</code> (optional chaining)

15.3 Django Debug Toolbar + React DevTools

React Developer Tools

```
# Установи расширение для Chrome:
# https://chrome.google.com/webstore/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoier

# После установки:
# F12 → вкладка "Components" — видишь дерево React-компонентов
# F12 → вкладка "Profiler" — измеряешь производительность
```

Проверка JWT в localStorage

```
# F12 → Console → введи:  
localStorage.getItem('access_token')  
# → "eyJhbGciOiJIUzI1NiIs..."  
  
localStorage.getItem('refresh_token')  
# → "eyJhbGciOiJIUzI1NiIs..."
```

Проверка запросов в Network tab

```
# F12 → Network → выбери запрос → посмотри:  
# - Headers: Authorization: Bearer eyJ... (токен подставился?)  
# - Payload: {email: "...", password: "..."} (данные верные?)  
# - Response: {access: "...", refresh: "..."} (токены вернулись?)  
# - Status: 200 (успех) или 401 (ошибка авторизации)
```

Часть 16. Что делать дальше

16.1 Улучшения: Tailwind, MUI, react-query

[DOC] [Tailwind CSS](#) · [MUI](#) · [TanStack Query](#)

Когда базовый функционал работает, можно улучшить:

Tailwind CSS – красивые стили без inline

```
# Установка:
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p

# Теперь вместо:
<div style={{ background: '#131921', color: 'white', padding: '10px 20px' }}>

# Пишешь:
<div className="bg-[#131921] text-white px-5 py-2.5">
```

MUI (Material UI) – готовые компоненты

```
# Установка:
npm install @mui/material @emotion/react @emotion/styled

# Использование:
import { Button, Card, TextField } from '@mui/material';

<Button variant="contained" color="warning">В корзину</Button>
<TextField label="Email" type="email" />
```

@tanstack/react-query – кэширование API

```
# Установка:
npm install @tanstack/react-query

// Преимущества:
// - Автоматическое кэширование (не загружаем одни и те же данные дважды)
// - Фоновое обновление (данные всегда свежие)
// - Optimistic updates (UI обновляется мгновенно, потом сервер подтверждает)
// - Retry при ошибке (автоматический повтор запроса)

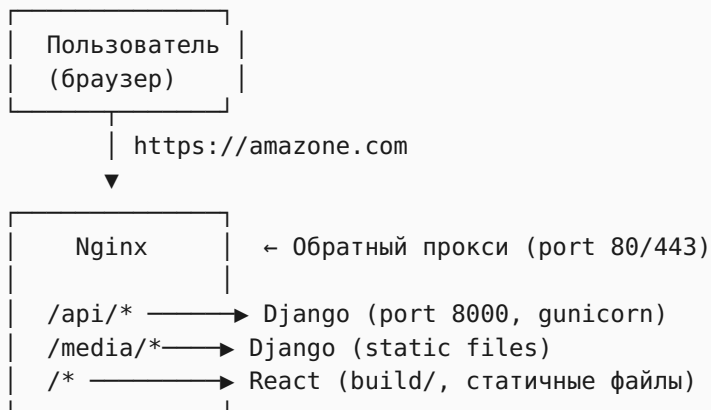
import { useQuery } from '@tanstack/react-query';

function ProductList() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['products'],
    queryFn: () => api.get(CATALOG.PRODUCTS).then(r => r.data),
    staleTime: 5 * 60 * 1000, // 5 минут — данные считаются свежими
  });
}
```

16.2 Deploy: как выложить в интернет

[DOC] [Django Deployment](#) · [Gunicorn](#) · [Nginx + Django](#)

Схема Production



Шаг 1: Собери React

```
cd frontend
npm run build
# Создаётся папка dist/ с оптимизированными файлами
```

Шаг 2: Настрой Nginx

```
# /etc/nginx/sites-available/amazone

server {
    listen 80;
    server_name amazone.com;

    # React (собранные файлы)
    location / {
        root /var/www/amazone/frontend/dist;
        try_files $uri $uri/ /index.html; # SPA: все маршруты → index.html
    }

    # Django API
    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # Django media
    location /media/ {
        proxy_pass http://127.0.0.1:8000;
    }

    # Django static (admin, etc.)
    location /static/ {
        proxy_pass http://127.0.0.1:8000;
    }
}
```

Шаг 3: Запусти Django через Gunicorn

```
pip install gunicorn
gunicorn config.wsgi:application --bind 0.0.0.0:8000 --workers 4
```

16.3 Чек-лист готовности к production

#	Пункт	Статус
1	DEBUG = False в settings.py	☐
2	SECRET_KEY из .env (не в коде!)	☐
3	ALLOWED_HOSTS настроен	☐
4	CORS_ALLOW_ALL_ORIGINS = False	☐
5	CORS_ALLOWED_ORIGINS = ваш домен	☐
6	PostgreSQL вместо SQLite	☐
7	HTTPS (SSL-сертификат через Let's Encrypt)	☐
8	Gunicorn вместо runserver	☐
9	Collectstatic + WhiteNoise	☐
10	Media-файлы через S3 или nginx	☐

Приложение А. Полная карта файлов проекта

I:\NewPythonProjects\

```

├── Amazone_Clone/                                ← Django-бэкенд
│   ├── config/
│   │   ├── settings.py        ← CORS, JWT, throttling, .env
│   │   ├── urls.py            ← Маршруты API + media + health
│   │   ├── test_runner.py     ← Кастомный runner для Python 3.14
│   │   └── wsgi.py
│   ├── apps/
│   │   ├── core/              ← BaseModel, health-check
│   │   ├── users/             ← User, Profile, Address, JWT login
│   │   ├── catalog/           ← Product, Category, Brand, Tag, Variant
│   │   ├── pricing/           ← Price, PriceHistory
│   │   ├── cart/              ← Cart, CartItem
│   │   ├── orders/            ← Order (FSM), OrderItem
│   │   ├── inventory/         ← Stock, StockMovement
│   │   ├── payments/          ← Payment, PaymentEvent
│   │   ├── reviews/          ← Review, ReviewImage
│   │   ├── discounts/         ← Campaign, Coupon
│   │   ├── shipping/          ← ShippingZone, Method, Shipment
│   │   ├── wishlist/          ← Wishlist, WishlistItem
│   │   ├── notifications/     ← Notification
│   │   └── analytics/         ← ProductView, AnalyticsService
│   ├── media/                 ← Картинки (dev)
│   └── manage.py
├── frontend/                                ← React-фронтенд
│   ├── src/
│   │   ├── api/
│   │   │   ├── client.ts      ← axios + JWT-интерцепторы
│   │   │   └── endpoints.ts   ← Все URL-пути
│   │   ├── stores/
│   │   │   ├── authStore.ts   ← Пользователь, логин, регистрация
│   │   │   ├── cartStore.ts   ← Корзина
│   │   │   ├── wishlistStore.ts ← Избранное
│   │   │   └── notificationStore.ts ← Уведомления
│   │   ├── types/
│   │   │   └── index.ts       ← TypeScript-интерфейсы
│   │   ├── components/
│   │   │   ├── Layout.tsx     ← Хедер + футер + корзина
│   │   │   └── ProtectedRoute.tsx ← Защита страниц
│   │   ├── pages/
│   │   │   ├── HomePage.tsx   ← /
│   │   │   ├── LoginPage.tsx  ← /login
│   │   │   ├── RegisterPage.tsx ← /register
│   │   │   ├── ProductListPage.tsx ← /products
│   │   │   ├── ProductDetailPage.tsx ← /products/:slug
│   │   │   ├── CartPage.tsx   ← /cart
│   │   │   ├── OrderCreatePage.tsx ← /checkout
│   │   │   ├── OrderListPage.tsx ← /orders
│   │   │   ├── OrderDetailPage.tsx ← /orders/:orderNumber
│   │   │   ├── ProfilePage.tsx ← /profile
│   │   │   └── WishlistPage.tsx ← /wishlist

```

```

|   ├── router.tsx      ← Маршруты
|   ├── main.tsx        ← Точка входа
|   └── index.css        ← Глобальные стили
├── index.html
├── package.json
├── tsconfig.json
└── vite.config.ts      ← Proxy на Django

```

Приложение В. Глоссарий (40 терминов)

Термин	Определение	Аналог в Django
SPA	Single Page Application – приложение с одной HTML-страницей, контент обновляется динамически	–
React	JavaScript-библиотека для построения UI. Рендерит компоненты в DOM	Django templates
TypeScript	JavaScript + типы. Компилируется в JavaScript перед запуском	mypy + type hints
Vite	Инструмент сборки. Dev-сервер + сборка для production	manage.py runserver
JSX	Синтаксис для написания HTML-подобного кода внутри JavaScript	Шаблонный язык Django
Component	Переиспользуемая единица UI. Функция, возвращающая JSX	template tag / include
Props	Данные, переданные от родительского компонента дочернему	context в template
State	Локальные данные компонента. Изменение → перерендер	–
useState	Хук для создания локального состояния	–
useEffect	Хук для побочных эффектов (запросы к API, подписки)	signal / middleware
Zustand	Библиотека глобального состояния. Проще Redux	request.user / session
Axios	HTTP-клиент для JavaScript. Аналог Python requests	requests / httpx
Interceptor	Функция, встраиваемая между запросом и отправкой (или между ответом и обработкой)	Django middleware
JWT	JSON Web Token – токен авторизации без сессий	Session auth
Access token	Короткоживущий JWT (15 мин). Прикрепляется к каждому запросу	sessionid cookie

Refresh token	Долгоживущий JWT (7 дней). Для получения нового access	—
localStorage	Хранилище в браузере. Данные сохраняются после закрытия вкладки	<code>django.contrib.sessions</code>
CORS	Cross-Origin Resource Sharing. Механизм безопасности браузера	—
Proxy	Перенаправление запросов. Vite проху: <code>/api → localhost:8000</code>	<code>nginx reverse proxy</code>
React Router	Библиотека для SPA-навигации без перезагрузки	<code>urls.py</code>
useParams	Хук для чтения динамических параметров URL	<code>kwargs</code> в <code>view</code>
useNavigate	Хук для программного перехода на другой URL	<code>redirect()</code>
Outlet	Компонент React Router, показывающий дочерний маршрут	<code>{% block content %}</code>
Navigate	Компонент для редиректа (аналог <code>redirect()</code> в Django)	<code>redirect()</code>
npm	Менеджер пакетов для JavaScript (Node Package Manager)	<code>pip</code>
package.json	Файл с зависимостями и скриптами проекта	<code>requirements.txt</code>
npm install	Установка всех зависимостей из <code>package.json</code>	<code>pip install -r requirements.txt</code>
npm run dev	Запуск dev-сервера Vite	<code>python manage.py runserver</code>
npm run build	Сборка проекта для production (оптимизация, минификация)	<code>python manage.py collectstatic</code>
TSX	TypeScript + JSX. Файлы компонентов React	<code>.html</code> шаблоны
Interface	TypeScript-тип, описывающий структуру объекта	<code>dataclass</code> / <code>TypedDict</code>
Generic	Обобщённый тип. <code>PaginatedResponse<Product></code>	Generic в <code>typing</code>

Optional chaining	<code>user?.name</code> – если <code>user=null</code> , вернёт <code>undefined</code> без ошибки	<code>getattr(user, 'name', None)</code>
Template literal	<code>`/products/\${slug}/`</code> – строка с подстановкой	<code>f"/products/{slug}/"</code>
API endpoint	URL-адрес, по которому фронтенд обращается к бэкенду	<code>path()</code> в <code>urls.py</code>
Pagination	Разделение списка на страницы (20 шт. на страницу)	Paginator в Django
Protected route	Страница, доступная только авторизованным	<code>@login_required</code>
Build	Результат <code>npm run build</code> . Оптимизированные HTML/JS/CSS	<code>collectstatic</code>
Hot Reload	Автообновление страницы при изменении кода	<code>runserver auto-reload</code>
Hook	Функция React (<code>useState</code> , <code>useEffect</code> , <code>useParams</code> и др.)	–

[PARTY]

Поздравляем!

Ты создал полноценный фронтенд для интернет-магазина.

Django-бэкенд: 14 модулей, 898 тестов, JWT, CORS, Swagger

React-фронтенд: 12 страниц, 4 store, API-клиент с autorefresh

Теперь ты понимаешь, как SPA-фронтенд общается с REST API бэкенда через HTTP-запросы и JSON.

Amazon Clone · Django + React · 2026